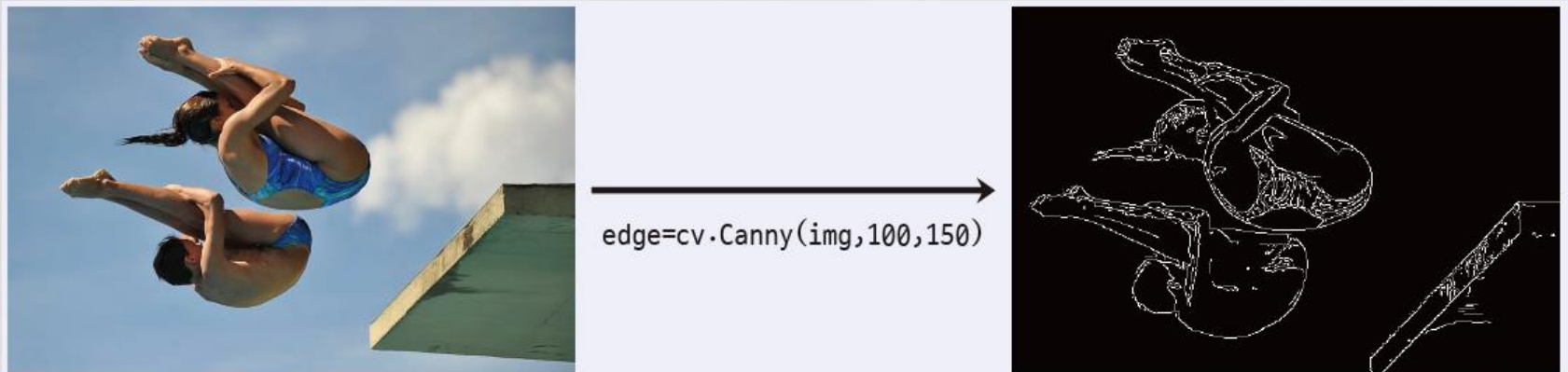


컴퓨터 비전

Preview

■ 현대는 양호한 프로그래밍 환경

- 예전에는 알고리즘을 바닥부터 구현
- 현대는 함수 호출로 영상 처리하는 시대. 대표적 컴퓨터 비전 라이브러리는 OpenCV



에지 검출이라는 큰 일을 거뜬히 수행하는 OpenCV의 Canny 함수

■ 파이썬 언어와 OpenCV 라이브러리로 컴퓨터 비전 세계를 활짝~~~

OpenCV 소개

■ 인텔이 만들어 공개한 OpenCV

- 바퀴를 다시 발명 reinventing the wheel 하는 쓸데없는 노력을 방지할 목적
- 인텔 칩의 성능을 평가할 목적

개요와 간략한 역사

■ 개요

- 클래스와 함수는 C와 C++로 개발. 전체 코드는 180만 라인 이상
- 인터페이스 언어는 C, C++, 자바, 자바스크립트, 파이썬
- OS 플랫폼은 윈도우, 리눅스, macOS, 안드로이드, iOS
- 교차 플랫폼 지원
- 교육과 상업 목적 모두 무료

개요와 간략한 역사

■ 간략 역사

OpenCV의 역사	
연도	사건
1998	• 인텔 직원인 개리 브라드스키(Gary Bradski)가 아이디어 제안
1999	• 오픈 소스로 공개하기로 결정하고 이름을 OpenCV로 정함
2000	• CVPR 컨퍼런스에서 알파 버전 발표
2001-2005	• 5개의 베타 버전 발표
2005	• 스탠퍼드 대학교의 자율주행차인 스탠리의 개발 팀에 합류해 그랜드 챌린지 우승 • OpenCV Korea 출범(https://cafe.naver.com/opencv)

개요와 간략한 역사

■ 간략 역사 (...계속)

2006	• OpenCV 1.0(C 인터페이스) 공개 •  로고 완성
2009	• OpenCV 2.0(C++ 인터페이스) 공개 • 파이썬과 자바 인터페이스 지원
2012	• 안드로이드와 iOS 지원 시작 • 깃허브로 마이그레이션
2015	• OpenCV 3.0 공개
2016	• 자바스크립트 인터페이스 지원 시작 • 딥러닝을 지원하는 DNN 모듈 추가
2018	• OpenCV 4.0 공개 • 고속 처리를 지원하는 OpenVINO 공개
2020	• Computer Vision and Deep Learning 코스 개설 • 전용 보드인 OpenCV AI Kit 출시
2022	• OpenCV 4.6 공개

공식 사이트

■ OpenCV를 지원하는 사이트

- 공식 홈페이지(<https://opencv.org>)
- 매뉴얼 사이트: 프로그래밍할 때 가장 많은 도움(<https://docs.opencv.org>)
- ...
- 깃허브
- 대한민국 OpenCV 사이트(<https://cafe.naver.com/opencv>)

공식 사이트

OpenCV: OpenCV modules

https://docs.opencv.org/4.5.5/index.html

ENHANCED BY Google

 **OpenCV**
Open Source Computer Vision

버전 선택

4.5.5

[Main Page](#) [Related Pages](#) [Modules](#) [Namespaces](#) [Classes](#) [Files](#) [Examples](#) [Java documentation](#)

Search

검색 창

OpenCV modules

- Introduction
- OpenCV Tutorials
- **OpenCV-Python Tutorials**
- OpenCV.js Tutorials
- Tutorials for contrib modules
- Frequently Asked Questions
- Bibliography
- Main modules:
 - core. Core functionality
 - imgproc. Image Processing
 - imgcodecs. Image file reading and writing
 - videoio. Video I/O
 - highgui. High-level GUI
 - video. Video Analysis
 - calib3d. Camera Calibration and 3D Reconstruction
 - features2d. 2D Features Framework
 - objdetect. Object Detection
 - dnn. Deep Neural Network module
 - ml. Machine Learning
 - flann. Clustering and Search in Multi-Dimensional Spaces
 - photo. Computational Photography
 - stitching. Images stitching
 - gapi. Graph API

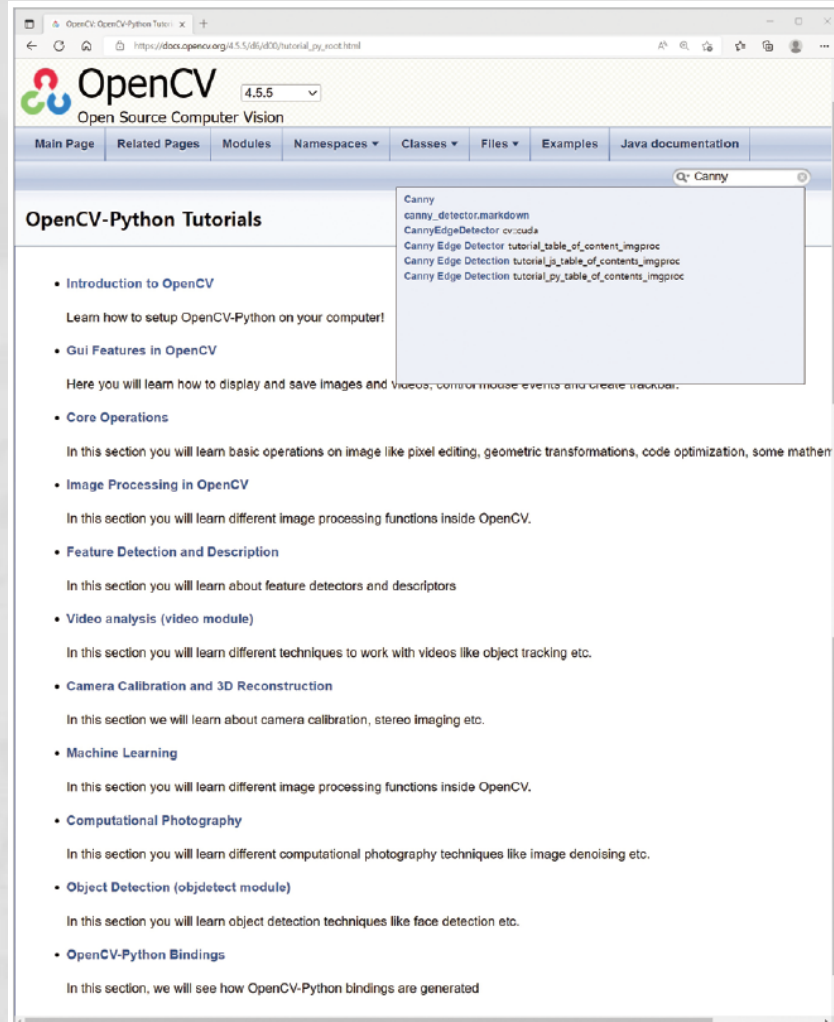
OpenCV 파이썬 튜토리얼

주요 모듈

OpenCV 매뉴얼 사이트

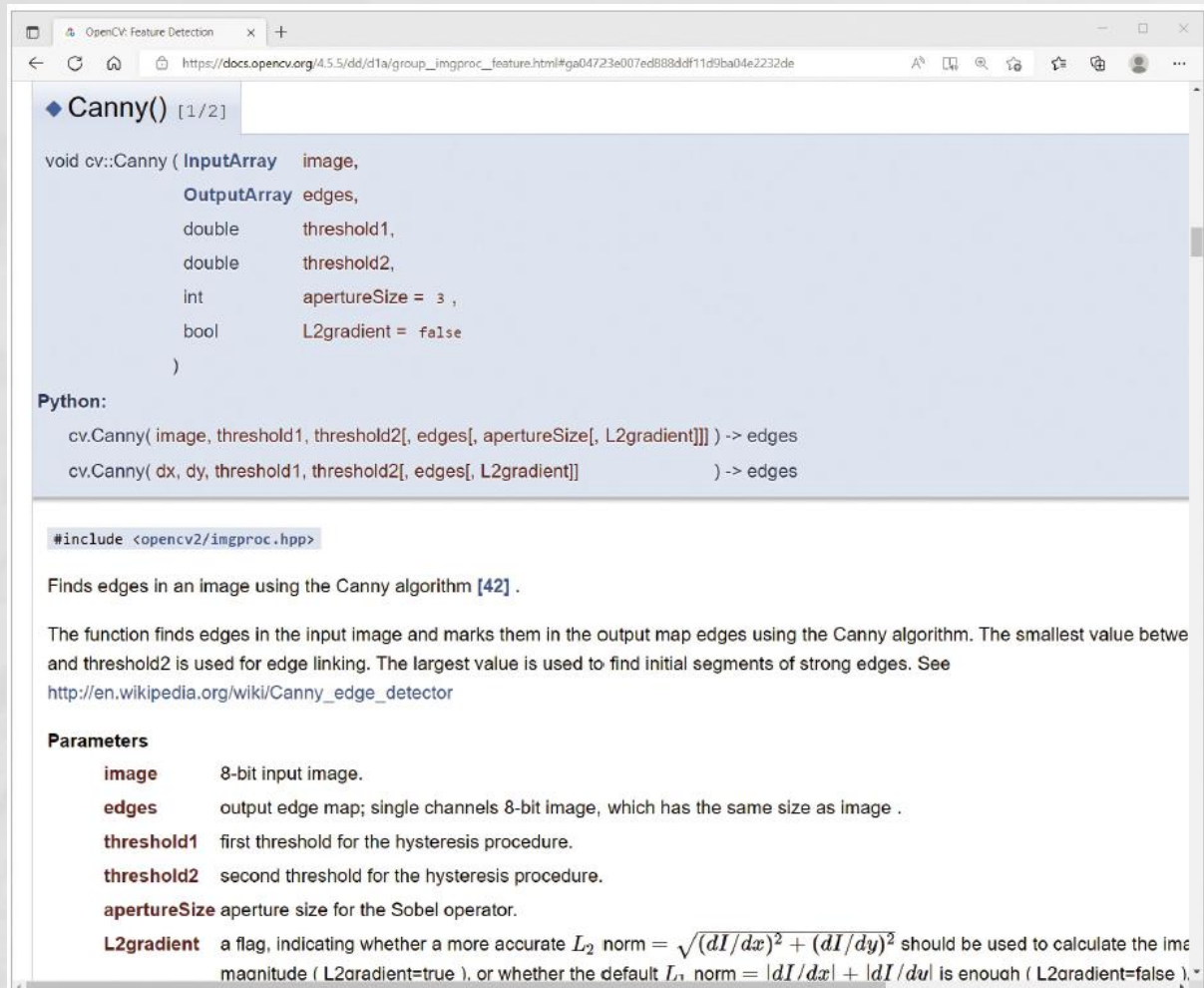
OpenCV 매뉴얼 활용하기

■ OpenCV-Python 튜토리얼을 잘 살필 것



OpenCV 매뉴얼 활용하기

■ 함수 선언을 잘 살필 것



The screenshot shows the OpenCV Canny() function documentation page. The browser address bar shows the URL: https://docs.opencv.org/4.5.5/dd/d1a/group_imgproc_feature.html#ga04723e007ed888ddf11d9ba04e2232de. The page title is "Canny() [1/2]".

C++ Declaration:

```
void cv::Canny ( InputArray image,
                OutputArray edges,
                double threshold1,
                double threshold2,
                int apertureSize = 3 ,
                bool L2gradient = false
            )
```

Python:

```
cv.Canny( image, threshold1, threshold2[, edges[, apertureSize[, L2gradient]]]) -> edges
cv.Canny( dx, dy, threshold1, threshold2[, edges[, L2gradient]] ) -> edges
```

Header:

```
#include <opencv2/imgproc.hpp>
```

Description:

Finds edges in an image using the Canny algorithm [42] .

The function finds edges in the input image and marks them in the output map edges using the Canny algorithm. The smallest value between threshold1 and threshold2 is used for edge linking. The largest value is used to find initial segments of strong edges. See http://en.wikipedia.org/wiki/Canny_edge_detector

Parameters

image	8-bit input image.
edges	output edge map; single channels 8-bit image, which has the same size as image .
threshold1	first threshold for the hysteresis procedure.
threshold2	second threshold for the hysteresis procedure.
apertureSize	aperture size for the Sobel operator.
L2gradient	a flag, indicating whether a more accurate L_2 norm = $\sqrt{(dI/dx)^2 + (dI/dy)^2}$ should be used to calculate the image gradient magnitude (L2gradient=true), or whether the default L_1 norm = $ dI/dx + dI/dy $ is enough (L2gradient=false),

객체지향 잘 활용하기

■ 파이썬은 객체지향 언어

- 객체지향은 컴퓨터 비전 프로그래밍에 매우 유리

객체 다루기

■ 객체지향 특성과 강점

- 객체는 능동적: 자신이 소유한 멤버 함수를 능동적으로 호출
- 필요한 만큼 얼마든지 찍어낼 수 있음



(a) a 객체



(b) b 객체



(c) c 객체

np.ndarray 클래스로 찍어낸 객체들

객체 확인하기 - type과 dir 내장 함수

■ 객체의 클래스를 알려주는 type과 사용 가능한 멤버 함수를 알려주는 dir

```
In [1]: type(a)
numpy.ndarray
In [2]: dir(a)
['T', '__abs__', '__add__', '__and__', ..., 'all', 'any', ..., 'max', ..., 'shape', ..., 'sort', ...]
```

■ 함수가 하는 일을 알아내려면 help

```
In [3]: help(a.sort)
Help on built-in function sort:

sort(...) method of numpy.ndarray instance
    a.sort(axis=-1, kind=None, order=None)

    Sort an array in-place. Refer to `numpy.sort` for full documentation.

    Parameters
    -----
    axis : int, optional
        Axis along which to sort. Default is -1, which means sort along the last
        axis.
```

영상을 읽고 표시하기

■ 처음 해보는 OpenCV 프로그래밍

영상 파일을 읽고 윈도우에 디스플레이하기

```
01 import cv2 as cv
02 import sys
03
04 img=cv.imread('soccer.jpg')    # 영상 읽기
05
06 if img is None:
07     sys.exit('파일을 찾을 수 없습니다.')
08
09 cv.imshow('Image Display',img) # 윈도우에 영상 표시
10
11 cv.waitKey()
12 cv.destroyAllWindows()
```



OpenCV에서 영상은 `numpy.ndarray` 클래스 형의 객체

■ `numpy`는 다차원 배열을 위한 사실상 표준 모듈

- 이런 이유로 OpenCV는 영상을 `numpy.ndarray`로 표현
- OpenCV가 다루는 영상은 `numpy`가 제공하는 다양한 기능(함수)을 사용할 수 있음

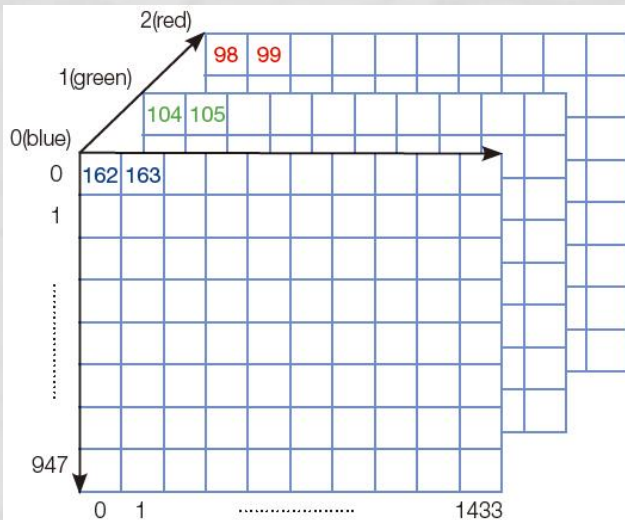
```
In [1]: type(img)
        numpy.ndarray
In [2]: img.shape
        (948,1434,3)
```

OpenCV에서 영상은 numpy.ndarray 클래스 형의 객체

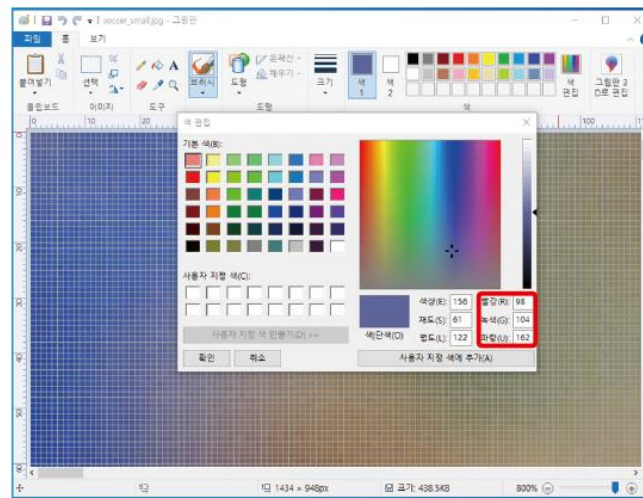
■ 영상의 표현

- 화소의 위치 (r,c) 또는 (y,x)
- 화소값 조사

```
In [3]: print(img[0,0,0], img[0,0,1], img[0,0,2])    # (0,0) 화소 조사  
162 104 98  
In [4]: print(img[0,1,0], img[0,1,1], img[0,1,2])    # (0,1) 화소 조사  
163 105 99
```



(a) 프로그램으로 조사



(b) 그림판으로 조사

img 객체가 표현하는 영상의 구조와 내용

영상 형태 변환하고 크기 축소하기

영상을 명암 영상으로 변환하고 반으로 축소하기

```
01 import cv2 as cv
02 import sys
03
04 img=cv.imread('soccer.jpg')
05
06 if img is None:
07     sys.exit('파일을 찾을 수 없습니다.')
08
09 gray=cv.cvtColor(img,cv.COLOR_BGR2GRAY) # BGR 컬러 영상을 명암 영상으로 변환
10 gray_small=cv.resize(gray,dsize=(0,0),fx=0.5,fy=0.5) # 반으로 축소
11
12 cv.imwrite('soccer_gray.jpg',gray) # 영상을 파일에 저장
13 cv.imwrite('soccer_gray_small.jpg',gray_small)
14
15 cv.imshow('Color image',img)
16 cv.imshow('Gray image',gray)
17 cv.imshow('Gray image small',gray_small)
18
19 cv.waitKey()
20 cv.destroyAllWindows()
```

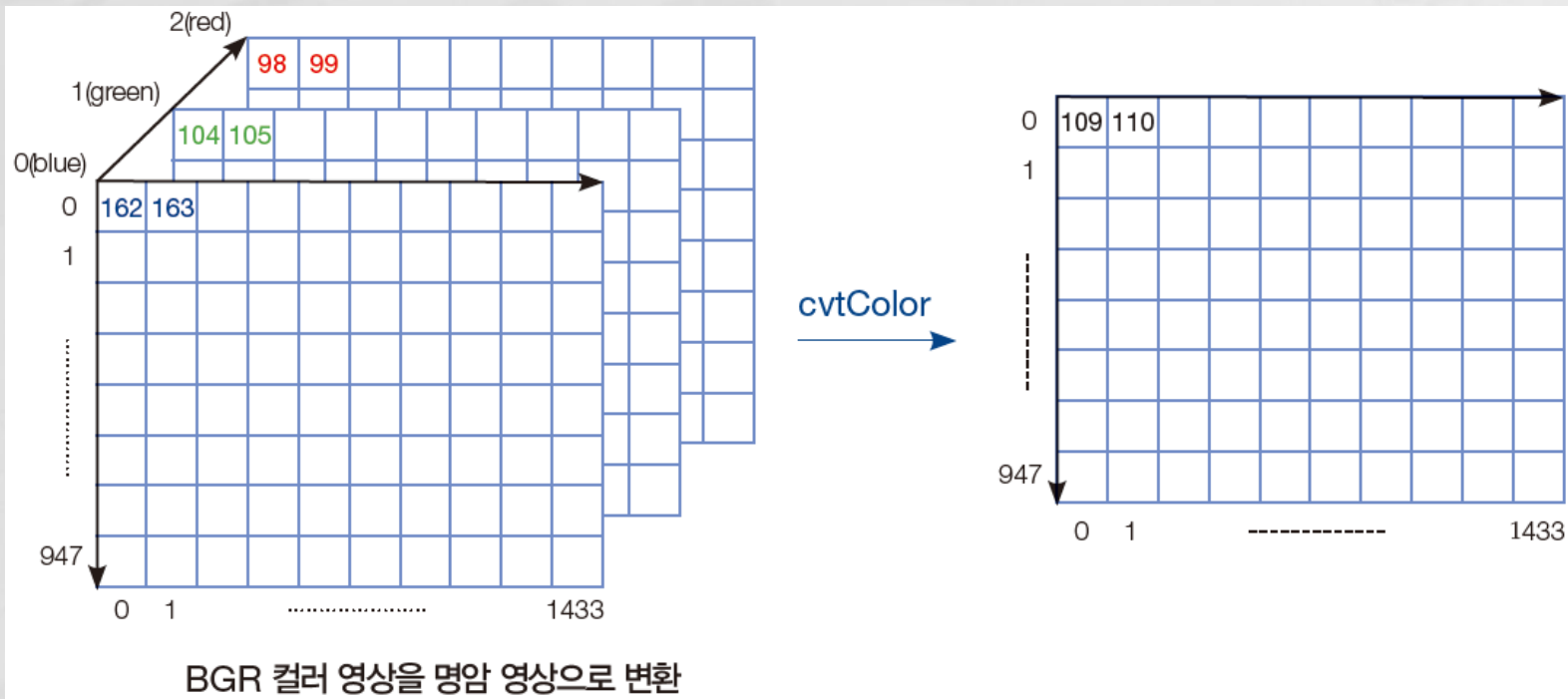


영상 형태 변환하고 크기 축소하기

■ cvtColor 함수가 컬러 영상을 명암 영상으로 바꾸는 방법

$$I = \text{round}(0.299 \times R + 0.587 \times G + 0.114 \times B) \quad (2.1)$$

(참조: https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html)



웹 캠에서 비디오 읽기

■ 웹 캠에서 비디오 읽기

웹 캠으로 비디오 획득하기

```
01 import cv2 as cv
02 import sys
03
04 cap=cv.VideoCapture(0,cv.CAP_DSHOW) # 카메라와 연결 시도
05
06 if not cap.isOpened():
07     sys.exit('카메라 연결 실패')
08
09 while True:
10     ret,frame=cap.read()          # 비디오를 구성하는 프레임 획득
11
12     if not ret:
13         print('프레임 획득에 실패하여 루프를 나갑니다.')
14         break
15
16     cv.imshow('Video display',frame)
17
18     key=cv.waitKey(1)              # 1밀리초 동안 키보드 입력 기다림
19     if key==ord('q'):              # 'q' 키가 들어오면 루프를 빠져나감
20         break
21
22 cap.release()                    # 카메라와 연결을 끊음
23 cv.destroyAllWindows()
```

웹 캠에서 비디오 읽기

■ 비디오에서 영상 수집하기(numpy의 hstack 함수 사용)

비디오에서 수집한 영상을 이어 붙이기

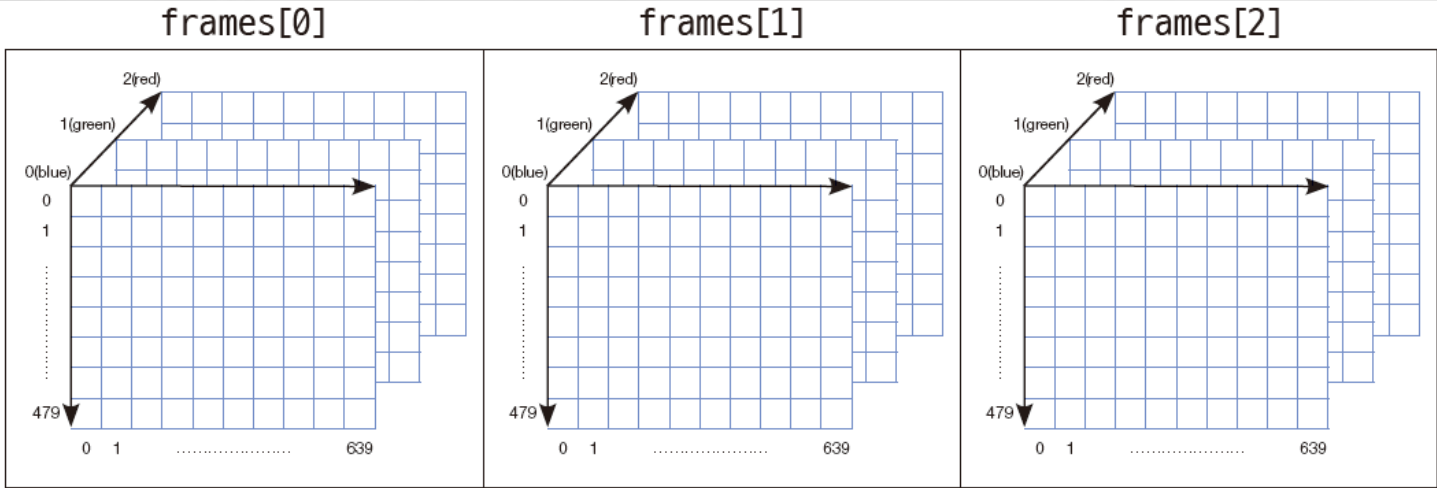
```
01 import cv2 as cv
02 import numpy as np
03 import sys
04
05 cap=cv.VideoCapture(0,cv.CAP_DSHOW)      # 카메라와 연결 시도
06
07 if not cap.isOpened():
08     sys.exit('카메라 연결 실패')
09
10 frames=[]
11 while True:
12     ret,frame=cap.read()                  # 비디오를 구성하는 프레임 획득
13
14     if not ret:
15         print('프레임 획득에 실패하여 루프를 나갑니다.')
16         break
17
```

웹 캠에서 비디오 읽기

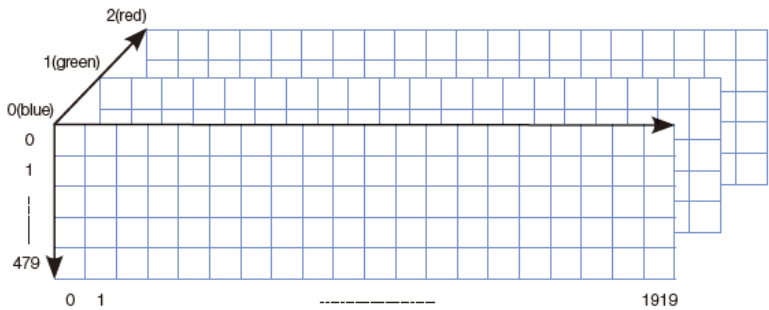
```
18     cv.imshow('Video display',frame)
19
20     key=cv.waitKey(1)                # 1밀리초 동안 키보드 입력 기다림
21     if key==ord('c'):                # 'c' 키가 들어오면 프레임을 리스트에 추가
22         frames.append(frame)
23     elif key==ord('q'):                # 'q' 키가 들어오면 루프를 빠져나감
24         break
25
26 cap.release()                        # 카메라와 연결을 끊음
27 cv.destroyAllWindows()
28
29 if len(frames)>0:                    # 수집된 영상이 있으면
30     imgs=frames[0]
31     for i in range(1,min(3,len(frames))): # 최대 3개까지 이어 붙임
32         imgs=np.hstack((imgs,frames[i]))
33
34     cv.imshow('collected images',imgs)
35
36     cv.waitKey()
37     cv.destroyAllWindows()
```

웹 캠에서 비디오 읽기

■ 자료구조



(a) frames 리스트



(b) imgs 배열

자료 구조

```
In [1]: len(frames)
7
In [2]: frames[0].shape
(480,640,3)
In [3]: type(imgs)
numpy.ndarray
In [4]: imgs.shape
(480,1920,3)
```

그래픽 기능과 사용자 인터페이스 만들기

■ OpenCV의 그래픽 기능

- 영상에 글씨나 도형을 넣는데 유용(Gui features in OpenCV 참조)
- line, rectangle, polylines, circle, ellipse, putText 함수

그래픽 기능과 사용자 인터페이스 만들기

■ 영상에 도형을 그리고 글씨 쓰기

영상에 도형을 그리고 글자 쓰기

```
01 import cv2 as cv
02 import sys
03
04 img=cv.imread('girl_laughing.jpg')
05
06 if img is None:
07     sys.exit('파일을 찾을 수 없습니다.')
08
09 cv.rectangle(img,(830,30),(1000,200),(0,0,255),2) # 직사각형 그리기
10 cv.putText(img,'laugh',(830,24),cv.FONT_HERSHEY_SIMPLEX,1,(255,0,0),2) # 글씨 쓰기
11
12 cv.imshow('Draw',img)
13
14 cv.waitKey()
15 cv.destroyAllWindows()
```



red

굵기

blue

그래픽 기능과 사용자 인터페이스 만들기



그래픽 기능과 사용자 인터페이스 만들기

■ 함수 선언에 대한 이해

◆ rectangle() [1/2]

```
void cv::rectangle ( InputOutputArray img,
                    Point pt1,
                    Point pt2,
                    const Scalar & color,
                    int thickness = 1,
                    int lineType = LINE_8,
                    int shift = 0
                  )
```

Python:

```
cv.rectangle( img, pt1, pt2, color[, thickness[, lineType[, shift]]] ) -> img
cv.rectangle( img, rec, color[, thickness[, lineType[, shift]]] ) -> img
```

```
#include <opencv2/imgproc.hpp>
```

Draws a simple, thick, or filled up-right rectangle.

The function `cv::rectangle` draws a rectangle outline or a filled rectangle whose two opposite corners are `pt1` and `pt2`.

Parameters

img	Image.
pt1	Vertex of the rectangle.
pt2	Vertex of the rectangle opposite to <code>pt1</code> .
color	Rectangle color or brightness (grayscale image).
thickness	Thickness of lines that make up the rectangle. Negative values, like <code>FILLED</code> , mean that the function has to draw a filled rectangle.
lineType	Type of the line. See LineTypes
shift	Number of fractional bits in the point coordinates.

OpenCV 공식 사이트가 제공하는 rectangle 함수의 선언(https://docs.opencv.org/4.x/d6/d6e/group__imgproc__draw.html#ga07d2f74cadcf8e305e810ce8eed13bc9)

그래픽 기능과 사용자 인터페이스 만들기

■ 마우스를 통한 상호작용(콜백 함수에 대한 이해 필요)

마우스로 클릭한 곳에 직사각형 그리기

```
01 import cv2 as cv
02 import sys
03
04 img=cv.imread('girl_laughing.jpg')
05
06 if img is None:
07     sys.exit('파일을 찾을 수 없습니다.')
08
09 def draw(event,x,y,flags,param):      # 콜백 함수
10     if event==cv.EVENT_LBUTTONDOWN:   # 마우스 왼쪽 버튼 클릭했
11         cv.rectangle(img,(x,y),(x+200,y+200),(0,0,255),2)
12     elif event==cv.EVENT_RBUTTONDOWN: # 마우스 오른쪽 버튼 클릭
13         cv.rectangle(img,(x,y),(x+100,y+100),(255,0,0),2)
14
15     cv.imshow('Drawing',img)
16
17 cv.namedWindow('Drawing')
18 cv.imshow('Drawing',img)
19
20 cv.setMouseCallback('Drawing',draw)   # Drawing 윈도우에 draw 콜백 함수 지정
21
22 while(True):                          # 마우스 이벤트가 언제 발생할지 모르므로 무한 반복
23     if cv.waitKey(1)==ord('q'):
24         cv.destroyAllWindows()
25     break
```



그래픽 기능과 사용자 인터페이스 만들기

■ 마우스 드래그로 도형 크기 조절하기

마우스 드래그로 직사각형 그리기

```
01 import cv2 as cv
02 import sys
03
04 img=cv.imread('girl_laughing.jpg')
05
06 if img is None:
07     sys.exit('파일을 찾을 수 없습니다.')
08
09 def draw(event,x,y,flags,param):
10     global ix,iy
11
12     if event==cv.EVENT_LBUTTONDOWN: # 마우스 왼쪽 버튼 클릭했을 때 초기 위치 저장
13         ix,iy=x,y
14     elif event==cv.EVENT_LBUTTONUP: # 마우스 왼쪽 버튼 클릭했을
15         cv.rectangle(img,(ix,iy),(x,y),(0,0,255),2)
16
17     cv.imshow('Drawing',img)
18
19 cv.namedWindow('Drawing')
20 cv.imshow('Drawing',img)
21
22 cv.setMouseCallback('Drawing',draw)
23
24 while(True):
25     if cv.waitKey(1)==ord('q'):
26         cv.destroyAllWindows()
27         break
```



페인팅

■ 페인팅 기능

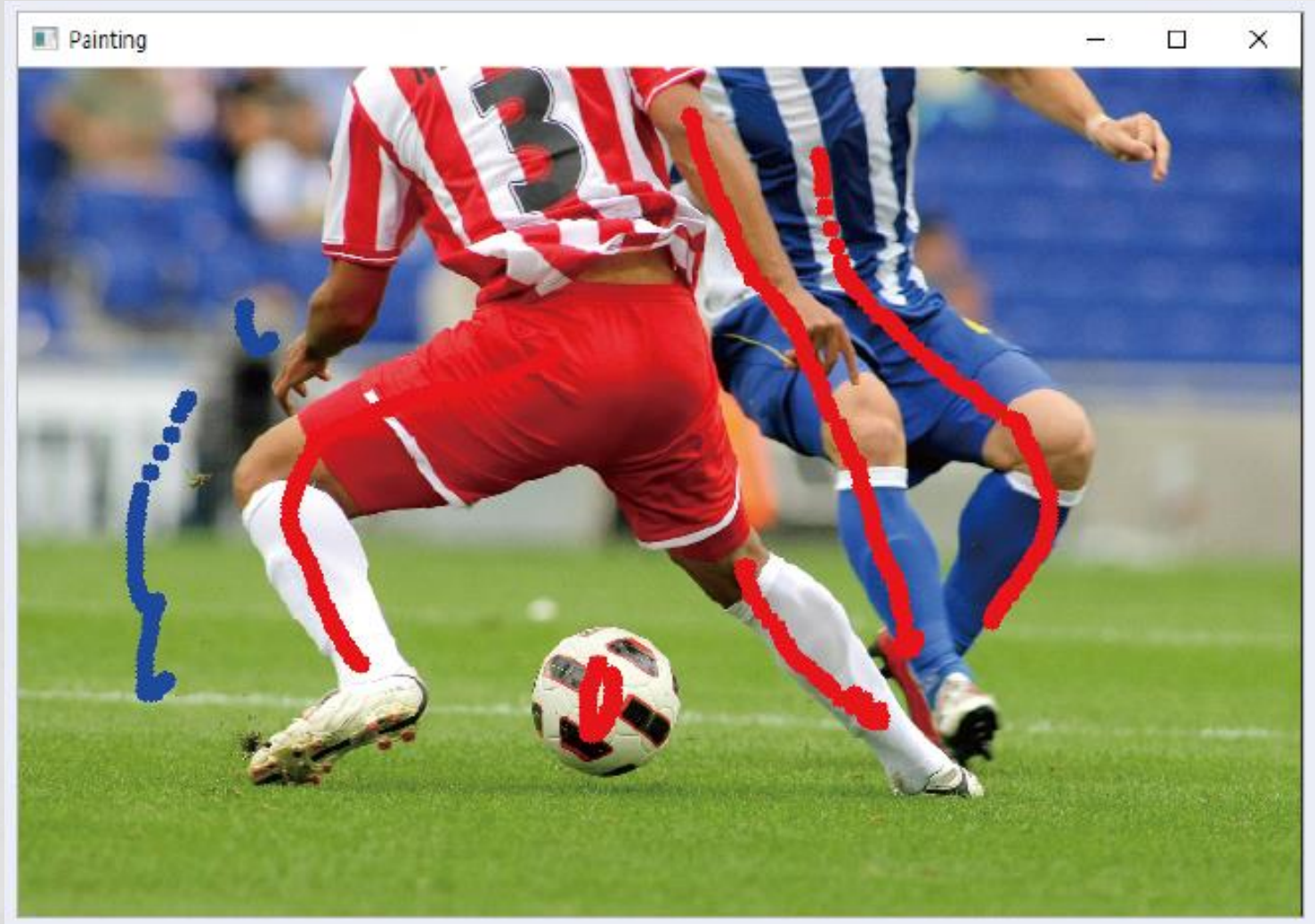
빨간색 붓과 파란색 붓으로 페인팅하기

```
01 import cv2 as cv
02 import sys
03
04 img=cv.imread('soccer.jpg')
05
06 if img is None:
07     sys.exit('파일을 찾을 수 없습니다.')
08
09 BrushSiz=5                                # 붓의 크기
10 LColor,RColor=(255,0,0),(0,0,255)        # 파란색과 빨간색
```

페인팅

```
11
12 def painting(event,x,y,flags,param):
13     if event==cv.EVENT_LBUTTONDOWN:
14         cv.circle(img,(x,y),BrushSiz,LColor,-1)    # 마우스 왼쪽 버튼 클릭하면 파란색
15     elif event==cv.EVENT_RBUTTONDOWN:
16         cv.circle(img,(x,y),BrushSiz,RColor,-1)    # 마우스 오른쪽 버튼 클릭하면 빨간색
17     elif event==cv.EVENT_MOUSEMOVE and flags==cv.EVENT_FLAG_LBUTTON:
18         cv.circle(img,(x,y),BrushSiz,LColor,-1)    # 왼쪽 버튼 클릭하고 이동하면 파란색
19     elif event==cv.EVENT_MOUSEMOVE and flags==cv.EVENT_FLAG_RBUTTON:
20         cv.circle(img,(x,y),BrushSiz,RColor,-1)    # 오른쪽 버튼 클릭하고 이동하면 빨간색
21
22     cv.imshow('Painting',img)                        # 수정된 영상을 다시 그림
23
24 cv.namedWindow('Painting')
25 cv.imshow('Painting',img)
26
27 cv.setMouseCallback('Painting',painting)
28
29 while(True):
30     if cv.waitKey(1)==ord('q'):
31         cv.destroyAllWindows()
32     break
```

페인팅



딥러닝 비전

Preview

■ 컴퓨터 비전의 오랜 난제와 딥러닝의 혁신



(a) 의미 분할 문제



(b) 영상 설명하기 문제

딥러닝이 일으킨 혁신

<https://huggingface.co/facebook/detr-resnet-50-panoptic>
https://huggingface.co/spaces/akhaliq/CLIP_prefix_captioning

방법론의 대전환

■ 규칙 기반 방법론과 딥러닝 방법론

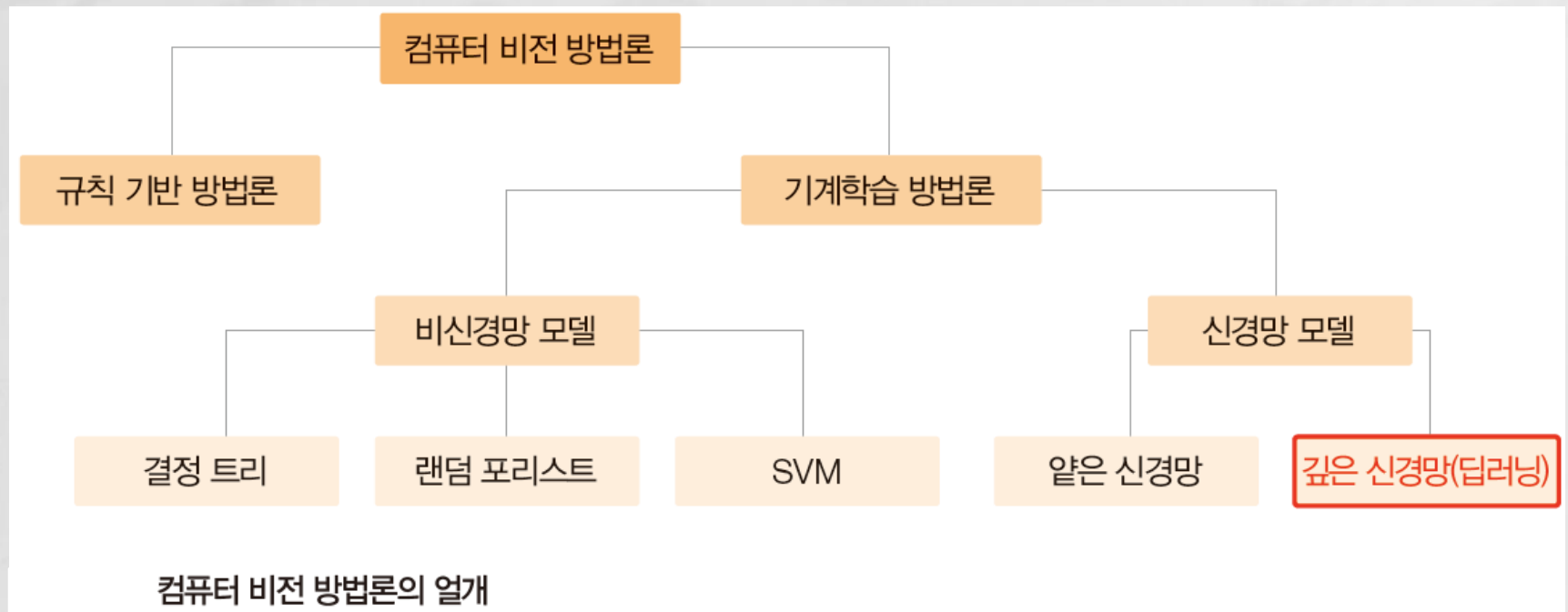
- 규칙 기반은 사람이 영상을 보고 규칙을 도출하고 프로그래밍
 - 분명한 한계. 예) 스케일 공간에서 가우시안보다 좋은 필터는 없을까? 데이터에 따라 최적 필터를 설계해야 하지 않을까?
- 딥러닝은 데이터 기반 학습을 통해 문제 해결
 - 예) 최적 필터를 학습으로 알아냄
 - 양질의 데이터가 매우 중요

■ 둘 다 이해하고 다룰 수 있을 때 컴퓨터 비전 전문성 완성

방법론의 대전환

■ 딥러닝으로 대전환함으로써 혁신을 이룸

- 신경망은 기계학습의 일종
- 딥러닝은 층을 깊게 쌓은 깊은 신경망을 이용



머신러닝 기초

■ 1단계: 데이터 수집

- 모델의 입력은 특징 벡터_{feature vector} 출력은 참값_{GT; ground truth} (또는 레이블_{label}이라 부름)

$$\text{특징 벡터: } \mathbf{x} = (x_1, x_2, \dots, x_d) \quad (7.2)$$

n 은 샘플의 개수(데이터셋 크기)

$$\text{데이터셋: } D = \{(\mathbf{x}^1, y^1), (\mathbf{x}^2, y^2), \dots, (\mathbf{x}^n, y^n)\} \quad (7.3)$$

특징 벡터 레이블(참값)

- 회귀_{regression}와 분류_{classification} 문제
 - 회귀는 레이블이 연속 값(온도, 판매량 등): 회귀 문제
 - 분류는 레이블이 이산 값(숫자 분류, 물체 분류, 성별 등)

머신러닝 기초

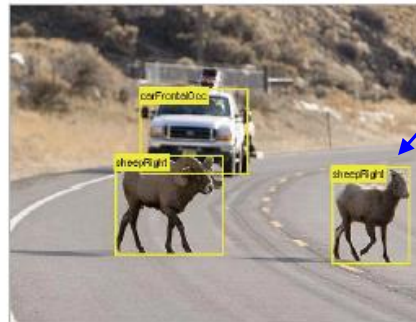
■ 1단계: 데이터 수집(...계속)

- 검출, 분할, 추적 등의 다양한 문제
- 데이터 수집은 많은 비용 소요
- 다행히 공개 데이터 많음

예: aihub 사이트(<https://www.aihub.or.kr/>)



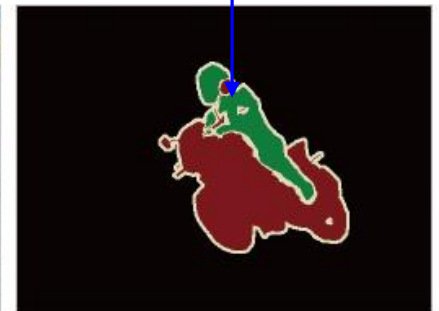
(a) 분류: Boat(왼쪽)와 Chair(오른쪽)



(b) 검출



(c) 분할



박스 집합이 레이블

화소 집합이 레이블

머신러닝 기초

■ 2단계: 모델 선택_{model selection}

- 선형 모델과 비선형 모델

- 예) 2차 함수를 모델로 사용하면 가중치(매개변수)가 3개로 확대

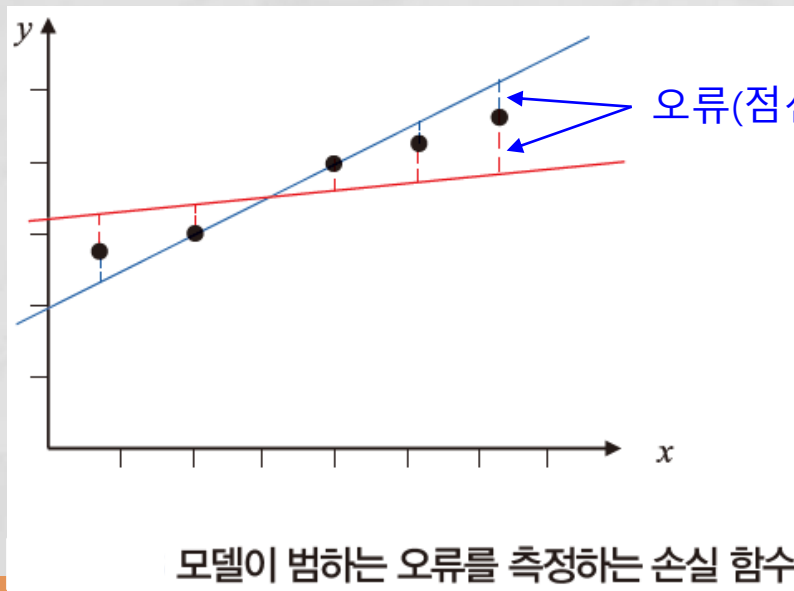
$$y = u_2 x^2 + u_1 x + u_0$$

- 가중치(매개변수) 집합 $\mathbf{w} = (u_0 \ u_1 \ u_2)$
- 딥러닝은 가중치가 수만~수조 개

머신러닝 기초

■ 3단계: 학습_{learning}

- 훈련 집합에 있는 샘플을 최소 오류로 맞추는 최적의 가중치 값을 알아내는 작업
- 단순한 경우, 방정식 풀어 해결하는 분석적 방법_{analytical method} 사용
- 기계학습은 오류를 조금씩 줄이는 과정을 반복하는 수치적 방법_{numerical method} 사용
- 모델이 범하는 오류를 측정하는 손실 함수_{loss function} 필요



머신러닝 기초

■ 3단계: 학습 (...계속)

- 평균제곱오차 MSE: mean squared error 는 가장 널리 쓰이는 손실 함수
 - 참값과 예측 값의 차이를 제곱하고 평균을 계산

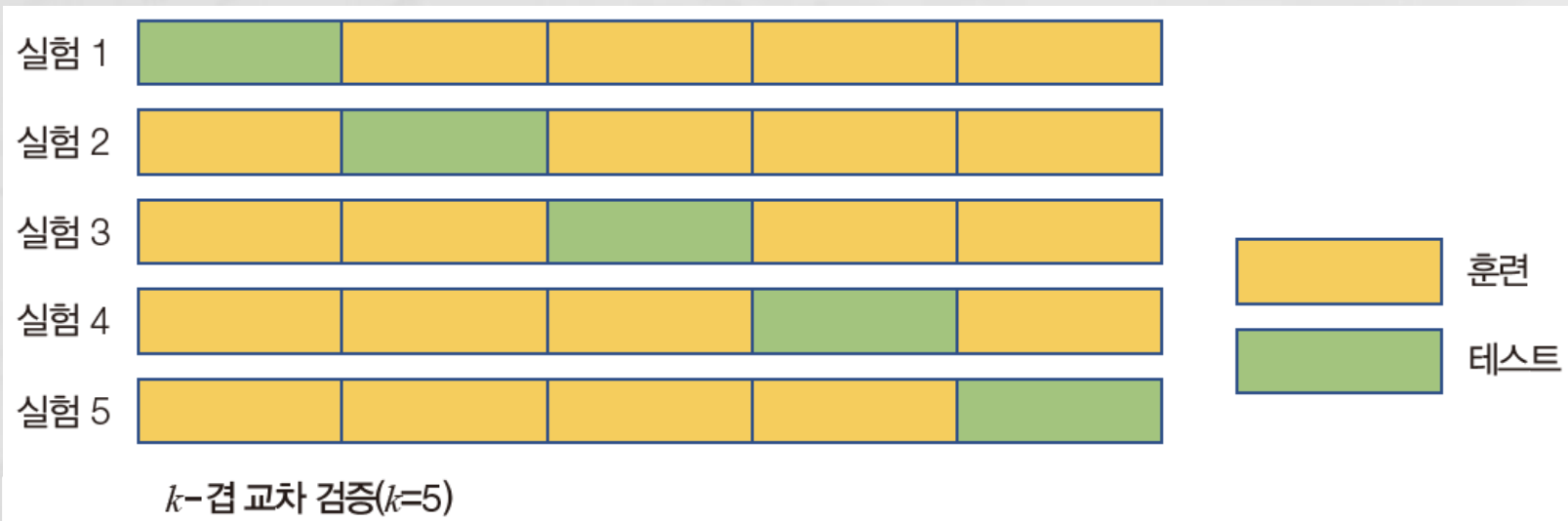
$$\text{평균제곱오차: } J(\mathbf{W}) = \frac{1}{n} \sum_{i=1, n} \left(f(\mathbf{x}^i) - y^i \right)^2 \quad (7.4)$$

- 손실 함수가 최소가 되는 점을 알아내는 최적화 알고리즘을 옵티마이저 optimizer라 부름
 - 파란색 모델이 최적에 더 가까움

머신러닝 기초

■ 4단계: 예측_{prediction} (또는 추론_{inference} 라고 부름)

- 학습을 마친 모델에 (학습에 사용하지 않았던) 새로운 특징 벡터를 입력하고 출력을 구하는 과정
- 예측을 통해 모델의 성능을 측정
- 일반화 능력 중요
 - 훈련 집합과 테스트 집합으로 나누고 테스트 집합으로 성능 측정
 - 또는 성능에 대한 신뢰를 높이려고 k-겹 교차 검증_{k-fold cross validation} 사용



딥러닝 소프트웨어 맛보기

■ 머신러닝 라이브러리

- sklearn은 딥러닝 이전의 고전적 기계학습 모델 지원(mlp, svm, decision tree, random forest 등)
- 텐서플로_{TensorFlow}와 파이토치_{PyTorch}는 딥러닝 지원

딥러닝 소프트웨어 맛보기

■ 텐서플로 소개

- 2015년 최초 공개
- 솔레는 텐서플로 위에서 동작하는 케라스를 개발하여 공개
- 2019년에 버전2를 공개: 버전2에서는 tensorflow 객체가 numpy와 호환되고 케라스와 한 몸이 됨(텐서플로 설치하면 케라스 따라 설치됨)

딥러닝 소프트웨어 맛보기

■ 데이터와 텐서

- 텐서플로가 제공하는 데이터셋 확인
 - MNIST 필기 숫자 데이터셋
 - CIFAR-10 자연영상 데이터셋
- 딥러닝에서는 다차원 배열을 텐서라 부름

텐서플로로 데이터 확인하기

```
01 import tensorflow as tf
02 import tensorflow.keras.datasets as ds
03 import matplotlib.pyplot as plt
04
05 (x_train,y_train),(x_test,y_test)=ds.mnist.load_data()
06 print(x_train.shape,y_train.shape,x_test.shape,y_test.shape) ①
07 plt.figure(figsize=(24,3))
08 plt.suptitle('MNIST',fontsize=30)
09 for i in range(10):
10     plt.subplot(1,10,i+1)
11     plt.imshow(x_train[i],cmap='gray') ②
12     plt.xticks([]); plt.yticks([])
13     plt.title(str(y_train[i]),fontsize=30)
```

딥러닝 소프트웨어 맛보기

```
15 (x_train,y_train),(x_test,y_test)=ds.cifar10.load_data()
16 print(x_train.shape,y_train.shape,x_test.shape,y_test.shape) ③
17 class_names=['airplane','car','bird','cat','deer','dog','frog','horse','ship',
    'truck']
18 plt.figure(figsize=(24,3))
19 plt.suptitle('CIFAR-10',fontsize=30)
20 for i in range(10):
21     plt.subplot(1,10,i+1)
22     plt.imshow(x_train[i]) ④
23     plt.xticks([]); plt.yticks([])
24     plt.title(class_names[y_train[i,0]],fontsize=30)
```

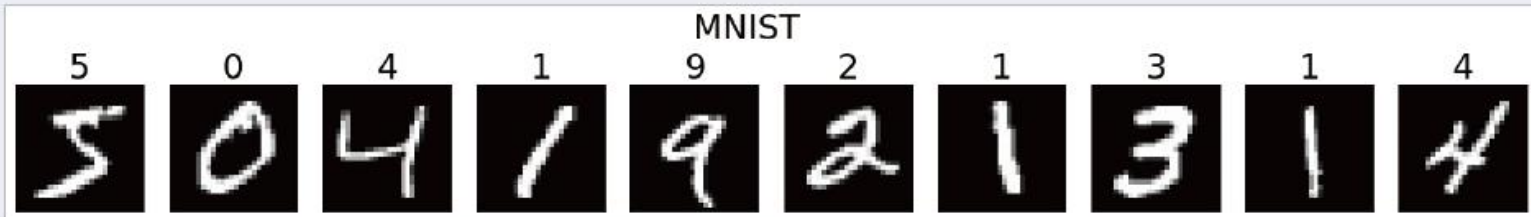
딥러닝 소프트웨어 맛보기

훈련 집합에 28*28 영상이 60000개

테스트 집합에 28*28 영상이 10000개

(60000, 28, 28) (60000,) (10000, 28, 28) (10000,) ①

MNIST



(50000, 32, 32, 3) (50000, 1) (10000, 32, 32, 3) (10000, 1) ③

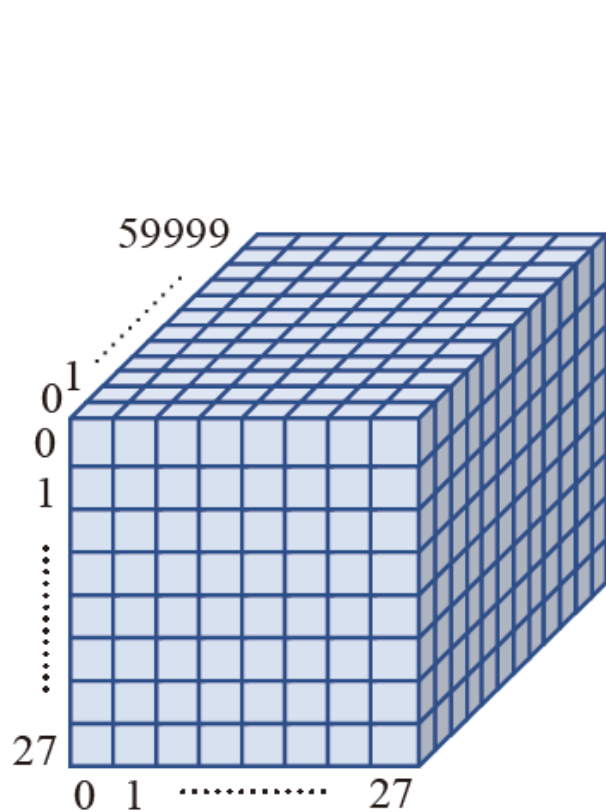
CIFAR-10



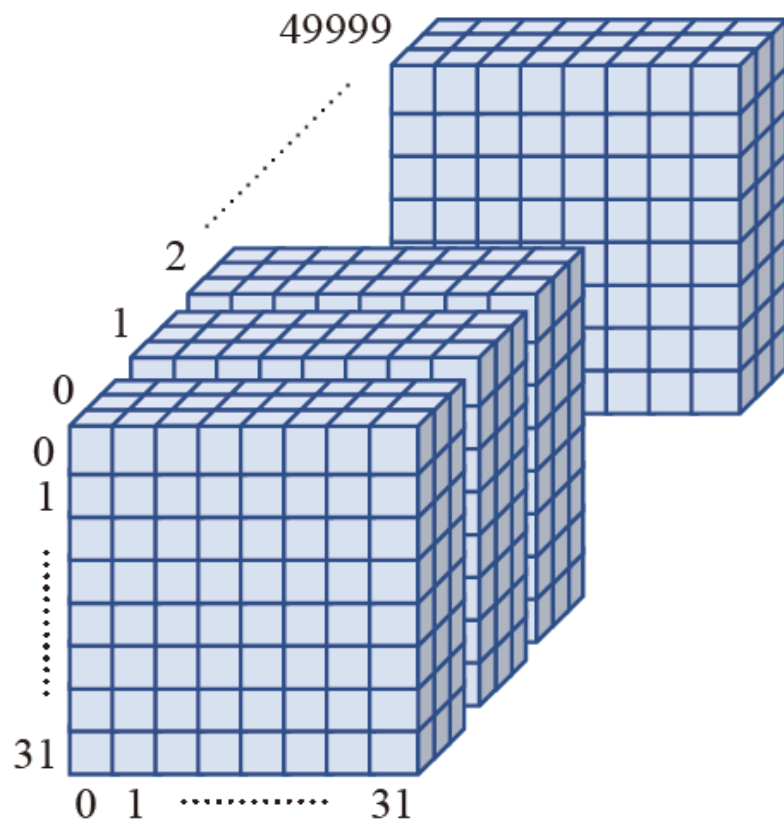
테스트 집합에 32*32*3 영상이 10000개

훈련 집합에 32*32*3 영상이 50000개

딥러닝 소프트웨어 맛보기



(a) MNIST의 x_train



(b) CIFAR-10의 x_train

데이터셋의 텐서 구조

퍼셉트론

■ 퍼셉트론

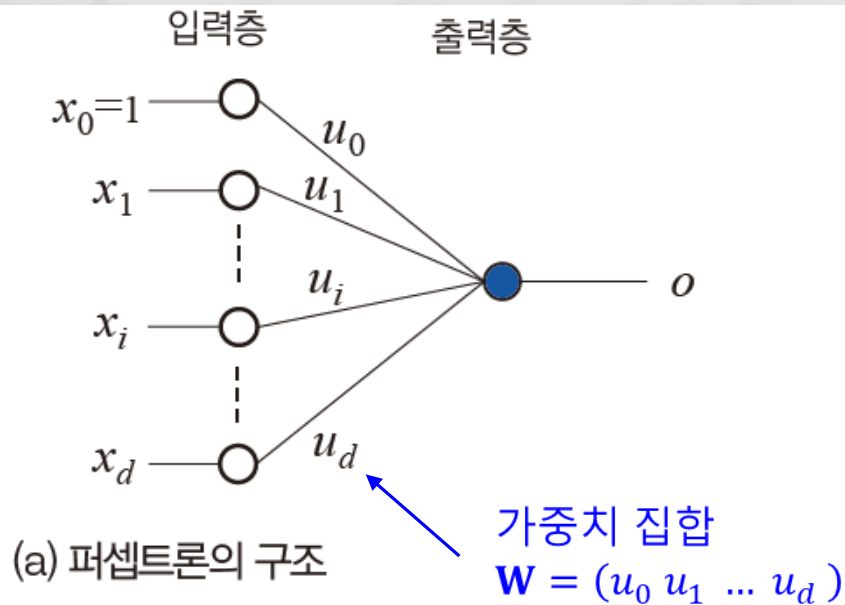
- 1958년에 등장한 원시적인 모델
- 딥러닝 이론의 토대이고 핵심 부품으로 사용되므로 딥러닝을 이해하는데 지름길

퍼셉트론

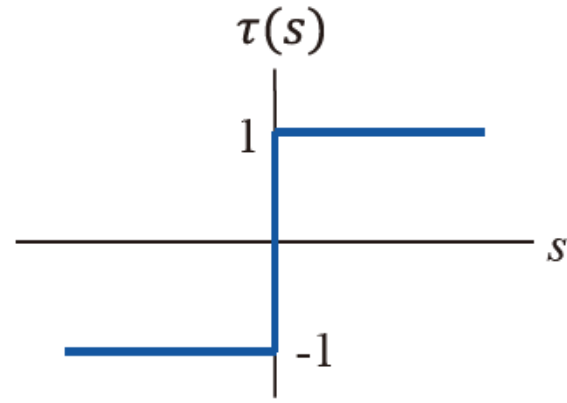
■ 구조와 연산

s 는 로짓(logit) τ 는 활성화 함수(activation function)

$$o = \tau(s) = \tau\left(\sum_{i=0,d} u_i x_i\right)$$
$$\tau(s) = \begin{cases} +1, s > 0 \\ -1, s \leq 0 \end{cases} \quad (7.5)$$



(a) 퍼셉트론의 구조



(b) 퍼셉트론이 사용하는 활성화 함수

퍼셉트론

■ 행렬 표기

- 샘플 하나 처리하는 상황
 - $\mathbf{x}$ 는 바이어스를 추가한 $(d+1)$ 차원 벡터

$$o = \tau(\mathbf{u}\mathbf{x}^T) \quad (7.6)$$

- 예) 두번째 샘플 $\mathbf{x}^2$ 에 대한 행렬 계산

$$o = \tau(\mathbf{u}\mathbf{x}^{2T}) = \tau\left((-0.5 \quad 1 \quad 1)\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}\right) = \tau(-0.5 \times 1 + 1 \times 0 + 1 \times 1) = \tau(0.5) = 1$$

- 훈련 집합 전체를 한꺼번에 처리하는 상황

$$\mathbf{O} = \tau(\mathbf{u}\mathbf{X}^T) \quad (7.7)$$

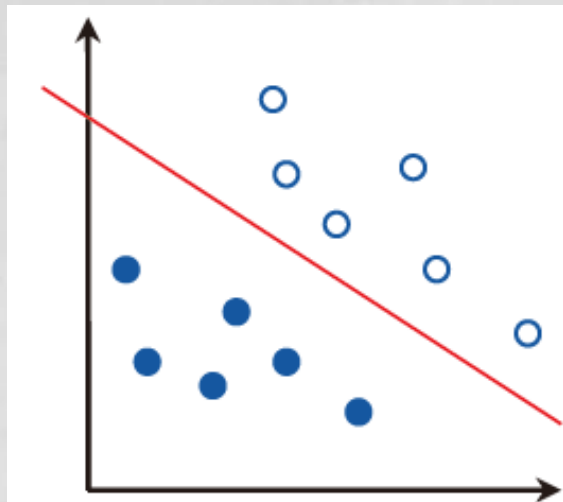


$\mathbf{x}$ 는 특징 벡터가 행에 배치된 설계 행렬(design matrix)

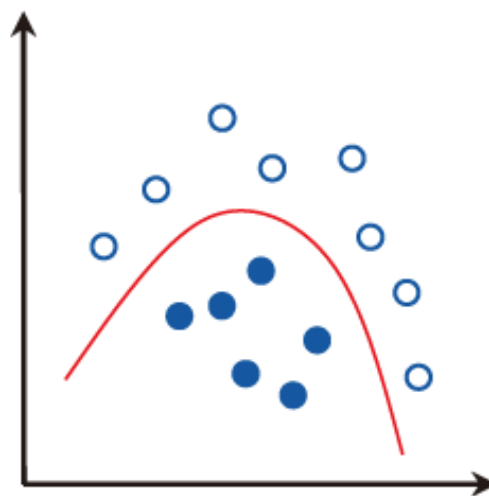
깊은 다층 퍼셉트론

■ 퍼셉트론은 선형 분류기

- 선형 분리 가능한 데이터만 높은 성능
- 현실에서는 대부분 비선형(선형 분리 불가능) 데이터



(a) 선형 분리 가능한 데이터셋



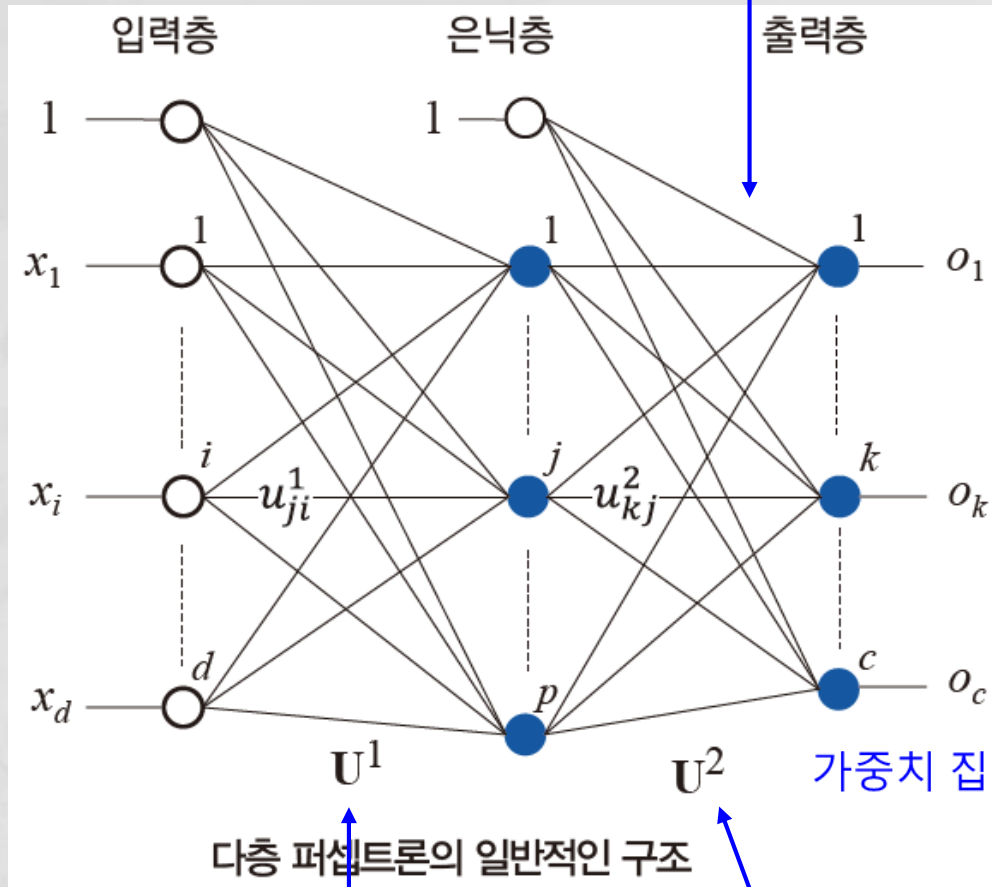
(b) 선형 분리 불가능한 데이터셋

선형 분리 가능과 불가능

■ 루멜하트는 다층 퍼셉트론으로 비선형 분류 문제 해결

다층 퍼셉트론

■ 다층 퍼셉트론의 구조



완전연결(FC: fully connected)

가중치 집합 $W = (U^1, U^2)$

다층 퍼셉트론의 일반적인 구조

은닉층과 출력층을 연결하는 가중치 행렬

입력층과 은닉층을 연결하는 가중치 행렬

다층 퍼셉트론

■ 가중치

- 은닉층의 j 번째 노드와 입력층의 i 번째 노드를 연결하는 에지 가중치를 u_{ji}^1 로 표기
- 출력층의 k 번째 노드와 은닉층의 j 번째 노드를 연결하는 에지 가중치를 u_{kj}^2 로 표기

$$\mathbf{U}^1 = \begin{pmatrix} u_{10}^1 & u_{11}^1 & \cdots & u_{1d}^1 \\ u_{20}^1 & u_{21}^1 & \cdots & u_{2d}^1 \\ \vdots & \vdots & \ddots & \vdots \\ u_{p0}^1 & u_{p1}^1 & \cdots & u_{pd}^1 \end{pmatrix} \quad \mathbf{U}^2 = \begin{pmatrix} u_{10}^2 & u_{11}^2 & \cdots & u_{1p}^2 \\ u_{20}^2 & u_{21}^2 & \cdots & u_{2p}^2 \\ \vdots & \vdots & \ddots & \vdots \\ u_{c0}^2 & u_{c1}^2 & \cdots & u_{cp}^2 \end{pmatrix} \quad (7.8)$$

■ 데이터에 따라 신경망 구조가 결정됨

- 특징 벡터의 차원이 d 이고 부류의 개수가 c 이면 입력층의 노드 개수는 $d + 1$ 이고 출력층의 노드 개수는 c
 - 예) MNIST 경우, 입력층은 785개($28 \times 28 + 1$), 출력층은 10개 노드를 가짐
- 은닉층의 노드 개수 p 는 하이퍼 매개변수(사용자가 지정해야 함)

다층 퍼셉트론

■ 전방 계산(노드 한 개를 위한 식)

- 신경망 구조가 복잡해져서 표기가 복잡해졌을 뿐 퍼셉트론의 식 (7.5)가 같음

$$j\text{번째 은닉 노드의 연산: } z_j = \tau\left(\sum_{i=0,d} u_{ji}^1 x_i\right) \quad (7.9)$$

- 식 (7.9)의 은닉 노드 계산을 행렬로 표기하면

$$j\text{번째 은닉 노드의 연산(행렬 표기): } z_j = \tau\left(\mathbf{u}_j^1 \mathbf{x}^T\right) \quad (7.10)$$

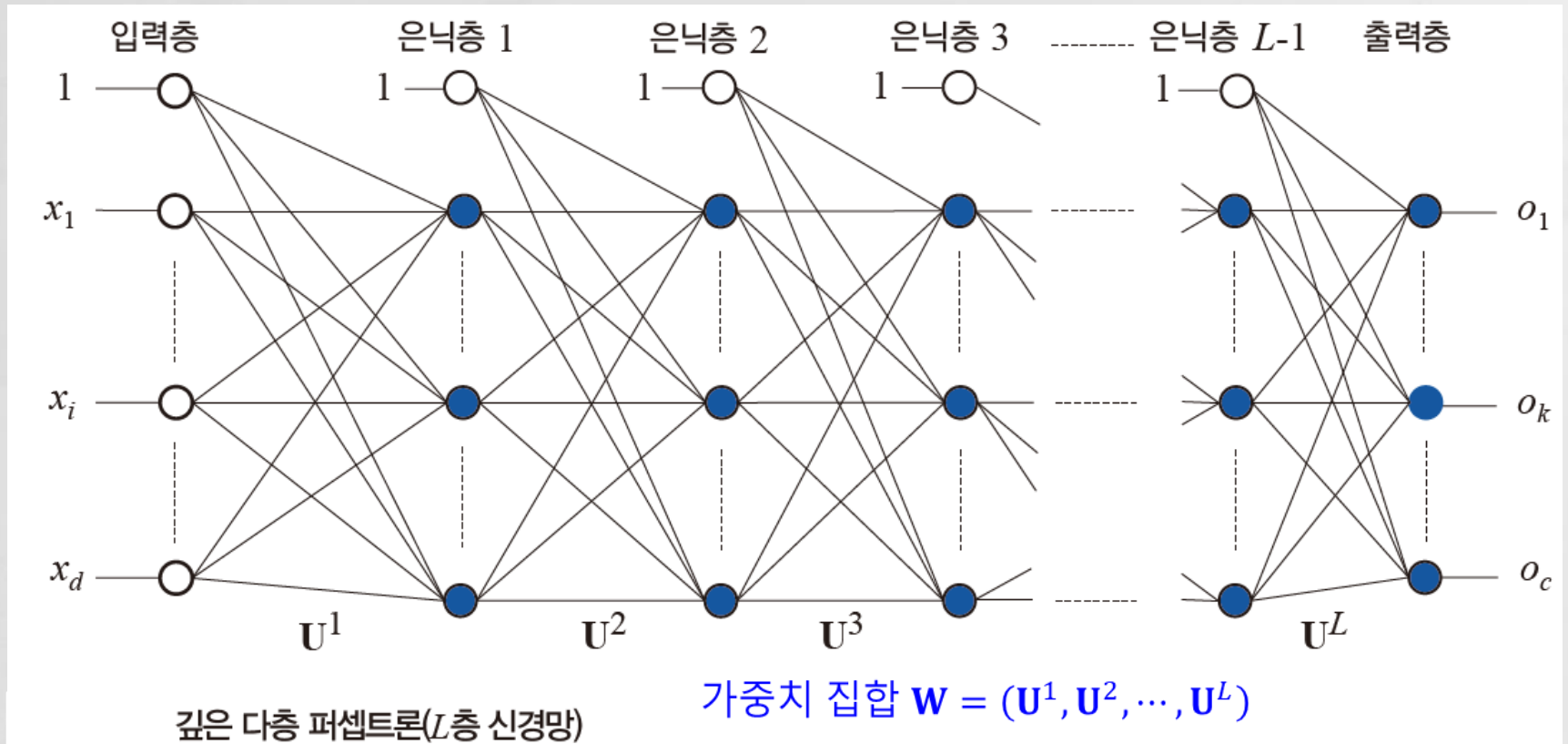
- $\mathbf{u}_j^1$ 은 $\mathbf{U}^1$ 의 j 번째 행 (즉 $\mathbf{u}_j^1 = (u_{j0}^1 \ u_{j1}^1 \ \cdots \ u_{jd}^1)$)
- 출력 노드 계산을 행렬로 표기하면

$$k\text{번째 출력 노드의 연산(행렬 표기): } o_k = \tau\left(\mathbf{u}_k^2 \mathbf{z}\right) \quad (7.11)$$

- $\mathbf{u}_k^2$ 은 $\mathbf{U}^2$ 의 k 번째 행 (즉 $\mathbf{u}_k^2 = (u_{k0}^2 \ u_{k1}^2 \ \cdots \ u_{kp}^2)$)

딥러닝의 서막: 깊은 다층 퍼셉트론

■ 깊은 다층 퍼셉트론의 전방 계산



딥러닝의 서막: 깊은 다층 퍼셉트론

■ l 번째 층의 가중치 행렬

- l 번째 층의 j 번째 노드와 $l-1$ 번째 층의 i 번째 노드를 연결하는 가중치를 u_{ji}^l 로 표기

$$\mathbf{U}^l = \begin{pmatrix} u_{10}^l & u_{11}^l & \cdots & u_{1n_{l-1}}^l \\ u_{20}^l & u_{21}^l & \cdots & u_{2n_{l-1}}^l \\ \vdots & \vdots & \ddots & \vdots \\ u_{n_l 0}^l & u_{n_l 1}^l & \cdots & u_{n_l n_{l-1}}^l \end{pmatrix}, \quad l=1,2,\cdots,L \quad (7.15)$$

딥러닝의 서막: 깊은 다층 퍼셉트론

- 전방 계산(샘플 하나를 위한 l 번째 층의 연산)

$$l\text{번째 층의 연산: } \mathbf{z}^l = \tau_l(\mathbf{U}^l \mathbf{z}^{l-1}), \quad l = 1, 2, \dots, L \quad (7.16)$$

- 전방 계산(샘플 하나를 위한 신경망 전체 연산)

$$\text{한 샘플의 전방 계산: } \mathbf{o} = \tau_L \left(\dots \tau_3 \left(\mathbf{U}^3 \tau_2 \left(\mathbf{U}^2 \tau_1 \left(\mathbf{U}^1 \mathbf{x}^T \right) \right) \right) \right) \quad (7.17)$$

- 전방 계산(설계 행렬을 위한 신경망 전체 연산)

$$\text{데이터셋 전체에 대한 전방 계산: } \mathbf{O} = \tau_L \left(\dots \tau_3 \left(\mathbf{U}^3 \tau_2 \left(\mathbf{U}^2 \tau_1 \left(\mathbf{U}^1 \mathbf{X}^T \right) \right) \right) \right) \quad (7.18)$$

딥러닝의 서막: 깊은 다층 퍼셉트론

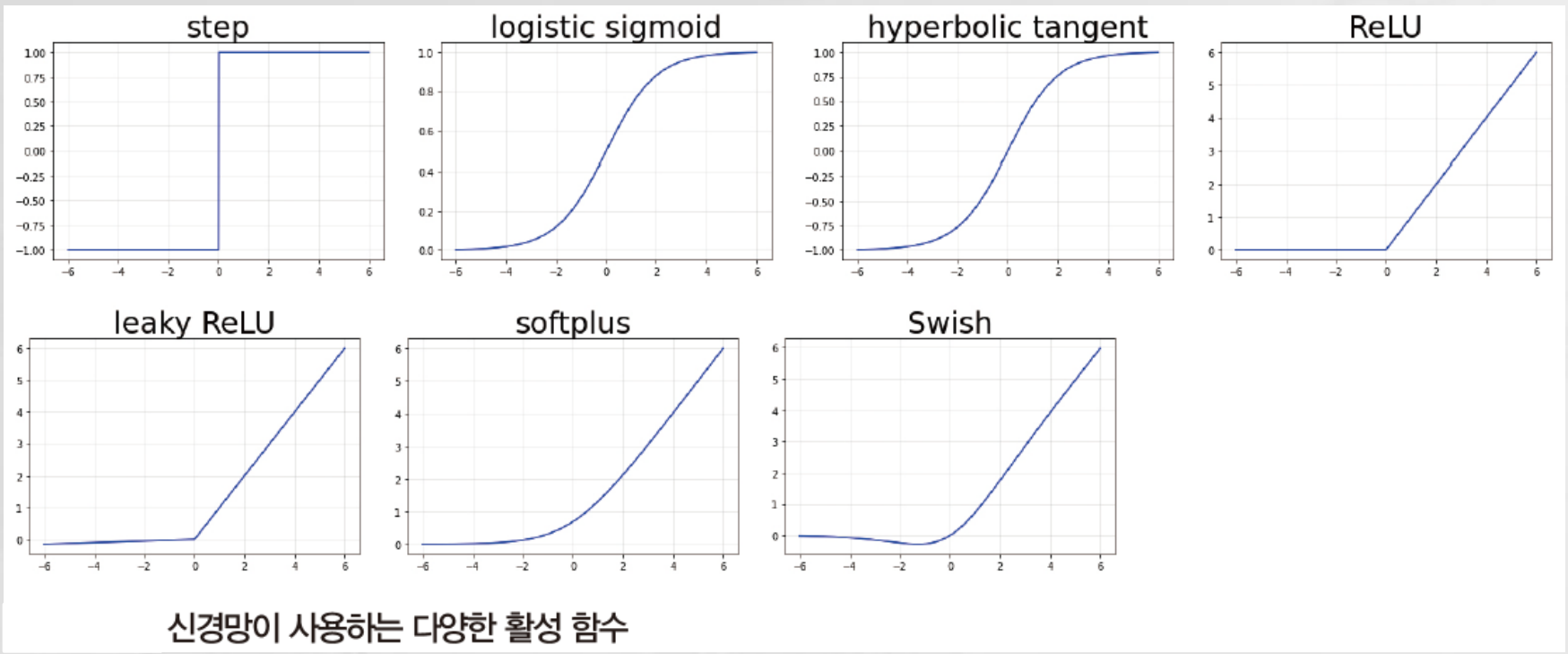
■ 신경망 출력을 부류 정보로 해석

- 이 단계를 예측_{prediction} 또는 추론_{inference}이라 부름

$$\text{최종 부류: } \hat{k} = \operatorname{argmax}_k o_k \quad (7.19)$$

- 출력층은 보통 softmax 활성 함수 사용

활성 함수



다양한 활성화 함수 $\tau(s)$

step	logistic sigmoid	hyperbolic tangent	ReLU	leaky LeRU	softplus	Swish
$\begin{cases} 1 & s > 0 \\ -1 & s \leq 0 \end{cases}$	$\frac{1}{1+e^{-\beta s}}$	$\frac{2}{1+e^{-\beta s}}-1$	$\max(0,s)$	$\begin{cases} s & s > 0 \\ 0.01s & s \leq 0 \end{cases}$	$\frac{\log(1+e^{\beta s})}{\beta}$	$\frac{s}{1+e^{-\beta s}}$

활성 함수

■ 출력층이 주로 사용하는 softmax

- 로짓 벡터 $\mathbf{s} = (s_1, s_2, \dots, s_c)$ 를 부류 확률 벡터 $\mathbf{o} = (o_1, o_2, \dots, o_c)$ 로 변환
- $o_1 + o_2 + \dots + o_c = 1$ 을 만족

$$o_k = \frac{e^{s_k}}{\sum_{i=1,c} e^{s_i}} \quad (7.20)$$

학습 알고리즘

■ 학습은

- 데이터셋 $D = \{\mathbf{X}, \mathbf{Y}\}$ 를 가장 잘 맞추는 가중치 $\mathbf{w} = (\mathbf{u}^1, \mathbf{u}^2, \dots, \mathbf{u}^L)$ 을 추정하는 문제

■ 훈련 데이터의 표현(MNIST와 CIFAR-10 데이터셋으로 설명)

- 특징 벡터 $\mathbf{x}^i$ 는 1차원 구조로 펼쳐서 표현
- 레이블 $\mathbf{y}^i$ 는 원핫 코드로 표현

$$\mathbf{x}^i = \text{[Handwritten digit 5 image]} = (x_1, x_2, \dots, x_{784})$$

$$\mathbf{y}^i = (0, 0, 0, 0, 0, 1, 0, 0, 0, 0)$$

(a) MNIST

$$\mathbf{x}^i = \text{[Ship image]} = (x_1, x_2, \dots, x_{3072})$$

$$\mathbf{y}^i = (0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0)$$

(b) CIFAR-10

학습에 사용되는 데이터 샘플의 벡터 표현

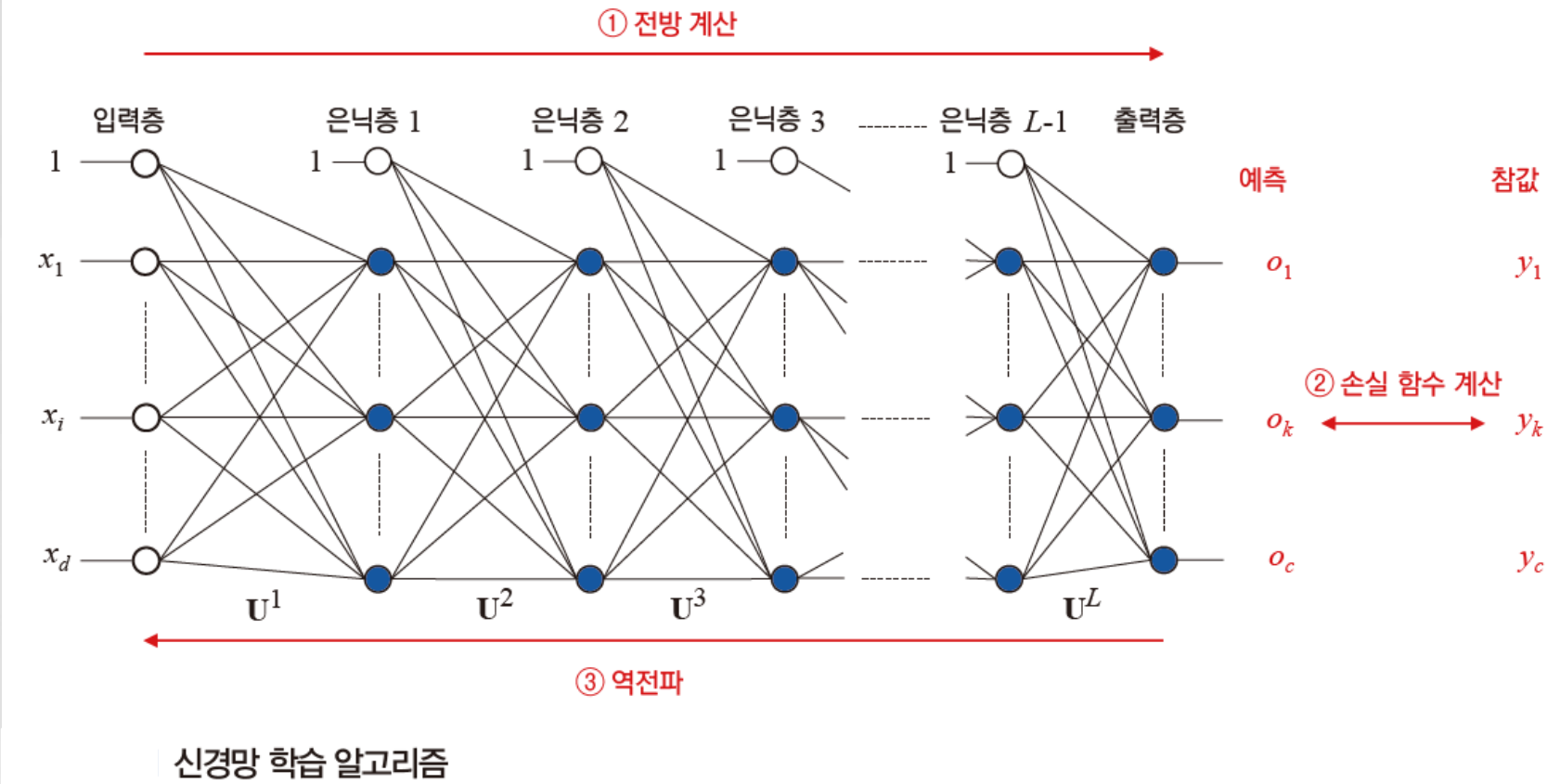
■ 학습 알고리즘이 알아내야 하는 가중치

- 예) MNIST를 위한 신경망
 - 은닉층 노드가 128개라면, 신경망은 $(784+1)*128+(128+1)*10 = 101,770$ 개의 가중치를 가짐
- 60,000개의 샘플로 높은 정확률로 맞히는 10만개 이상의 가중치를 알아내는 최적화 문제

■ 학습 전략

- 처음에는 가중치에 대한 실마리가 없기 때문에 난수 설정하고 출발
- 손실 함수 값을 줄이는 방향으로 가중치 갱신을 반복
- 충분히 높은 정확률을 달성하거나 더 이상 개선이 불가능하다고 여겨질 때 멈춤

스토캐스틱 경사하강법



다층 퍼셉트론 구현하기

- 다층 퍼셉트론과 깊은 다층 퍼셉트론을 실습
 - MNIST와 CIFAR-10 데이터셋 사용

필기 숫자 인식

다층 퍼셉트론으로 MNIST 인식하기(SGD 옵티마이저)

```
01 import numpy as np
02 import tensorflow as tf
03 import tensorflow.keras.datasets as ds
04
05 from tensorflow.keras.models import Sequential
06 from tensorflow.keras.layers import Dense
07 from tensorflow.keras.optimizers import SGD
08
09 (x_train,y_train),(x_test,y_test)=ds.mnist.load_data()
10 x_train=x_train.reshape(60000,784) [0,1]로 정규화
11 x_test=x_test.reshape(10000,784)
12 x_train=x_train.astype(np.float32)/255.0
13 x_test=x_test.astype(np.float32)/255.0
14 y_train=tf.keras.utils.to_categorical(y_train,10)
15 y_test=tf.keras.utils.to_categorical(y_test,10)
16
```

1차원 구조로 변환

데이터 준비

원핫 코드로 변환

필기 숫자 인식

은닉층 노드 개수 512, 출력층 노드 개수 10

은닉층 활성화 함수 tanh, 출력층 활성화 함수 softmax

```
17 mlp=Sequential()
18 mlp.add(Dense(units=512,activation='tanh',input_shape=(784,)))
19 mlp.add(Dense(units=10,activation='softmax'))
20     MSE 손실 함수   SGD 옵티마이저           학습률
21 mlp.compile(loss='MSE',optimizer=SGD(learning_rate=0.01),
22             metrics=['accuracy'])
23     ↑             ↑             ↑
24 res=mlp.evaluate(x_test,y_test,verbose=0)
25 print('정확률=',res[1]*100) ②
```

모델 선택
(신경망
구조 설계)

학습

예측(성능 측정)

훈련 집합

미니 배치 크기

세대 수

필기 숫자 인식

Epoch 1/50 ①

469/469 - 1s - loss: 0.0876 - accuracy: 0.2390 - val_loss: 0.0842 - val_accuracy:
0.3410 - 1s/epoch - 3ms/step

Epoch 2/50

469/469 - 1s - loss: 0.0805 - accuracy: 0.4025 - val_loss: 0.0764 - val_accuracy:
0.4418 - 1s/epoch - 2ms/step

...

Epoch 49/50

469/469 - 1s - loss: 0.0189 - accuracy: 0.8870 - val_loss: 0.0179 - val_accuracy:
0.8941 - 1s/epoch - 2ms/step

Epoch 50/50

469/469 - 1s - loss: 0.0187 - accuracy: 0.8874 - val_loss: 0.0178 - val_accuracy:
0.8946 - 1s/epoch - 2ms/step

정확률= 89.4599974155426 ②

필기 숫자 인식

■ Adam 옵티마이저를 사용하여 성능 향상

- 21행에서 optimizer 인수를 Adam으로 바꿈

다층 퍼셉트론으로 MNIST 인식하기(Adam 옵티마이저)

```
07  from tensorflow.keras.optimizers import Adam

21  mlp.compile(loss='MSE',optimizer=Adam(learning_rate=0.001),metrics
    =['accuracy'])
```

필기 숫자 인식

빠른 수렴

```
Epoch 1/50
469/469 - 1s - loss: 0.0148 - accuracy: 0.9028 - val_loss: 0.0109 - val_accuracy:
0.9284 - 1s/epoch - 3ms/step
Epoch 2/50
469/469 - 1s - loss: 0.0089 - accuracy: 0.9428 - val_loss: 0.0073 - val_accuracy:
0.9523 - 1s/epoch - 2ms/step
...
Epoch 49/50
469/469 - 1s - loss: 2.4108e-04 - accuracy: 0.9987 - val_loss: 0.0029 - val_accuracy:
0.9822 - 1s/epoch - 2ms/step
Epoch 50/50
469/469 - 1s - loss: 2.8139e-04 - accuracy: 0.9984 - val_loss: 0.0029 - val_accuracy:
0.9819 - 1s/epoch - 2ms/step
정확률= 98.18999767303467
```

SGD에 비해 무려 8.73% 향상

성능 시각화

■ SGD와 Adam 성능을 그래프로 비교

다층 퍼셉트론으로 MNIST 인식하기(SGD와 Adam의 성능 그래프 비교)

```
01 import numpy as np
02 import tensorflow as tf
03 import tensorflow.keras.datasets as ds
04
05 from tensorflow.keras.models import Sequential
06 from tensorflow.keras.layers import Dense
07 from tensorflow.keras.optimizers import SGD, Adam
08
09 (x_train,y_train),(x_test,y_test)=ds.mnist.load_data()
10 x_train=x_train.reshape(60000,784)
11 x_test=x_test.reshape(10000,784)
12 x_train=x_train.astype(np.float32)/255.0
13 x_test=x_test.astype(np.float32)/255.0
14 y_train=tf.keras.utils.to_categorical(y_train,10)
15 y_test=tf.keras.utils.to_categorical(y_test,10)
16
17 mlp_sgd=Sequential()
18 mlp_sgd.add(Dense(units=512,activation='tanh',input_shape=(784,)))
19 mlp_sgd.add(Dense(units=10,activation='softmax'))
```

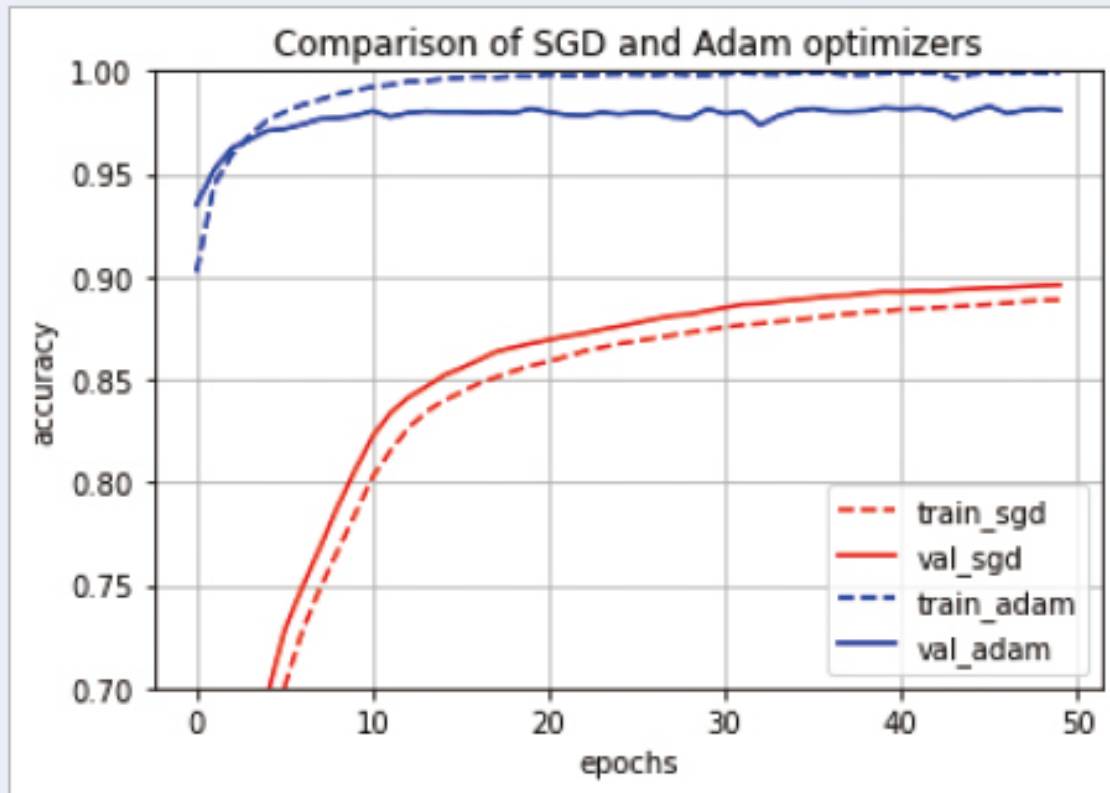
성능 시각화

```
20
21 mlp_sgd.compile(loss='MSE',optimizer=SGD(learning_rate=0.01),metrics=['accuracy'])
22 hist_sgd=mlp_sgd.fit(x_train,y_train,batch_size=128,epochs=50,validation_
    data=(x_test,y_test),verbose=2)
23 print('SGD 정확률=',mlp_sgd.evaluate(x_test,y_test,verbose=0)[1]*100)
24
25 mlp_adam=Sequential()
26 mlp_adam.add(Dense(units=512,activation='tanh',input_shape=(784,)))
27 mlp_adam.add(Dense(units=10,activation='softmax'))
28
29 mlp_adam.compile(loss='MSE',optimizer=Adam(learning_rate=0.001),metrics
    =['accuracy'])
30 hist_adam=mlp_adam.fit(x_train,y_train,batch_size=128,epochs=50,validation_
    data=(x_test,y_test),verbose=2)
31 print('Adam 정확률=',mlp_adam.evaluate(x_test,y_test,verbose=0)[1]*100)
32
33 import matplotlib.pyplot as plt
34
35 plt.plot(hist_sgd.history['accuracy'],'r--')
36 plt.plot(hist_sgd.history['val_accuracy'],'r')
37 plt.plot(hist_adam.history['accuracy'],'b--')
38 plt.plot(hist_adam.history['val_accuracy'],'b')
39 plt.title('Comparison of SGD and Adam optimizers')
40 plt.ylim((0.7,1.0))
41 plt.xlabel('epochs')
42 plt.ylabel('accuracy')
43 plt.legend(['train_sgd','val_sgd','train_adam','val_adam'])
44 plt.grid()
45 plt.show()
```


성능 시각화

SGD 정확률= 89.60000276565552

Adam 정확률= 98.07999730110168



하이퍼 매개변수 다루기

■ 하이퍼 매개변수

- 신경망의 구조 또는 학습과 관련하여 사용자가 설정해야 하는 매개변수
- 아주 많은 하이퍼 매개변수가 있음
- 예) 층 수, 노드 수, 활성화 함수, 옵티마이저, 학습률, 미니배치 크기, 세대 수, 손실 함수 ...

■ 예) Dense 층과 Adam 옵티마이저의 API

좋은 값을 기본값으로 설정해 둬

```
tf.keras.layers.Dense(units, activation=None, use_bias=True, kernel_initializer="glorot_uniform", bias_initializer="zeros", kernel_regularizer=None, bias_regularizer=None, activity_regularizer=None, kernel_constraint=None, bias_constraint=None, **kwargs)
```

```
tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, epsilon=1e-07, amsgrad=False, name="Adam", **kwargs)
```

하이퍼 매개변수 다루기

- 하이퍼 매개변수 설정에 만능은 없음. 몇 가지 유용한 요령
 - 텐서플로가 제공하는 기본값 사용
 - 신뢰할 수 있는 논문이나 웹 사이트가 제공하는 권고 사항 따름
 - 중요한 하이퍼 매개변수 1~3개에 대해 성능 실험을 통해 스스로 설정

하이퍼 매개변수 다루기

■ 깊은 다층 퍼셉트론으로 MNIST 인식

깊은 다층 퍼셉트론으로 MNIST 인식하기

```
01 import numpy as np
02 import tensorflow as tf
03 import tensorflow.keras.datasets as ds
04
05 from tensorflow.keras.models import Sequential
06 from tensorflow.keras.layers import Dense
07 from tensorflow.keras.optimizers import Adam
08
09 (x_train,y_train),(x_test,y_test)=ds.mnist.load_data()
10 x_train=x_train.reshape(60000,784)
11 x_test=x_test.reshape(10000,784)
12 x_train=x_train.astype(np.float32)/255.0
13 x_test=x_test.astype(np.float32)/255.0
14 y_train=tf.keras.utils.to_categorical(y_train,10)
15 y_test=tf.keras.utils.to_categorical(y_test,10)
16
17 dmlp=Sequential()
18 dmlp.add(Dense(units=1024,activation='relu',input_shape=(784,)))
19 dmlp.add(Dense(units=512,activation='relu'))
20 dmlp.add(Dense(units=512,activation='relu'))
21 dmlp.add(Dense(units=10,activation='softmax'))
```

신경망의 층을 4개로 늘림



하이퍼 매개변수 다루기

학습률을
0.0001로 설정

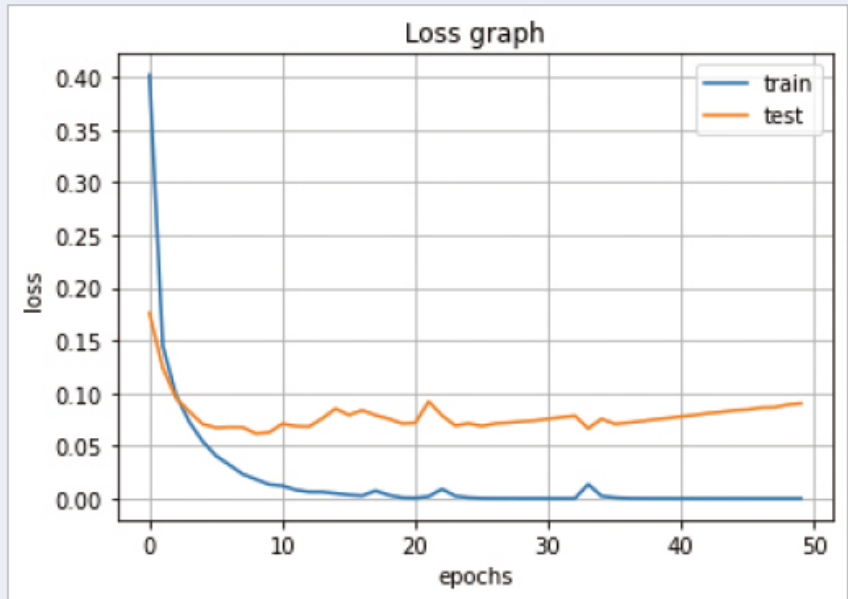
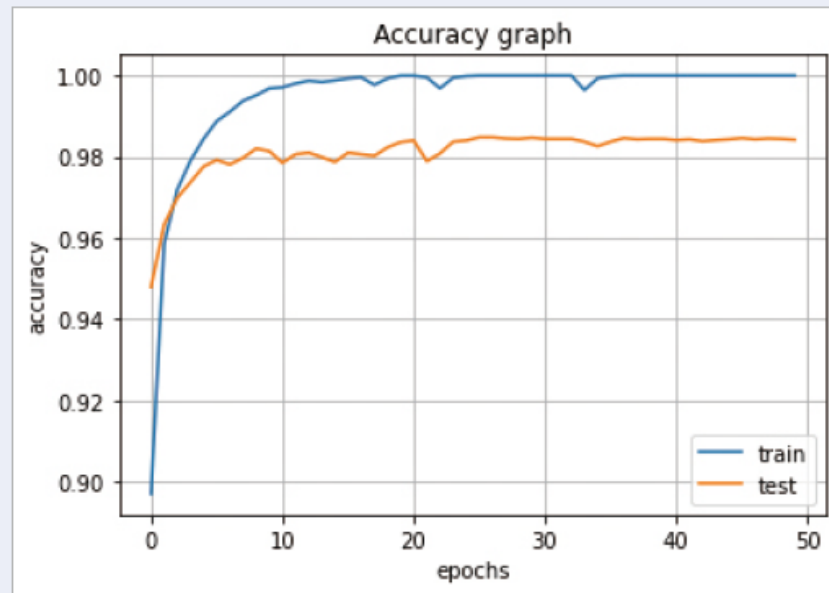
```
22
23 dmlp.compile(loss='categorical_crossentropy',optimizer=Adam(learning_rate
    =0.0001),metrics=['accuracy'])
24 hist=dmlp.fit(x_train,y_train,batch_size=128,epochs=50,validation_data=(x_
    test,y_test),verbose=2)
25 print('정확률=', dmlp.evaluate(x_test,y_test,verbose=0)[1]*100)
26
27 dmlp.save('dmlp_trained.h5')
28
29 import matplotlib.pyplot as plt
30
31 plt.plot(hist.history['accuracy'])
32 plt.plot(hist.history['val_accuracy'])
33 plt.title('Accuracy graph')
34 plt.xlabel('epochs')
35 plt.ylabel('accuracy')
36 plt.legend(['train','test'])
37 plt.grid()
38 plt.show()
39
40 plt.plot(hist.history['loss'])
41 plt.plot(hist.history['val_loss'])
42 plt.title('Loss graph')
43 plt.xlabel('epochs')
44 plt.ylabel('loss')
45 plt.legend(['train','test'])
46 plt.grid()
47 plt.show()
```

교차 엔트로피 손실 함수 사용

비전 에이전트 제작을 위해
학습된 모델을 저장

하이퍼 매개변수 다루기

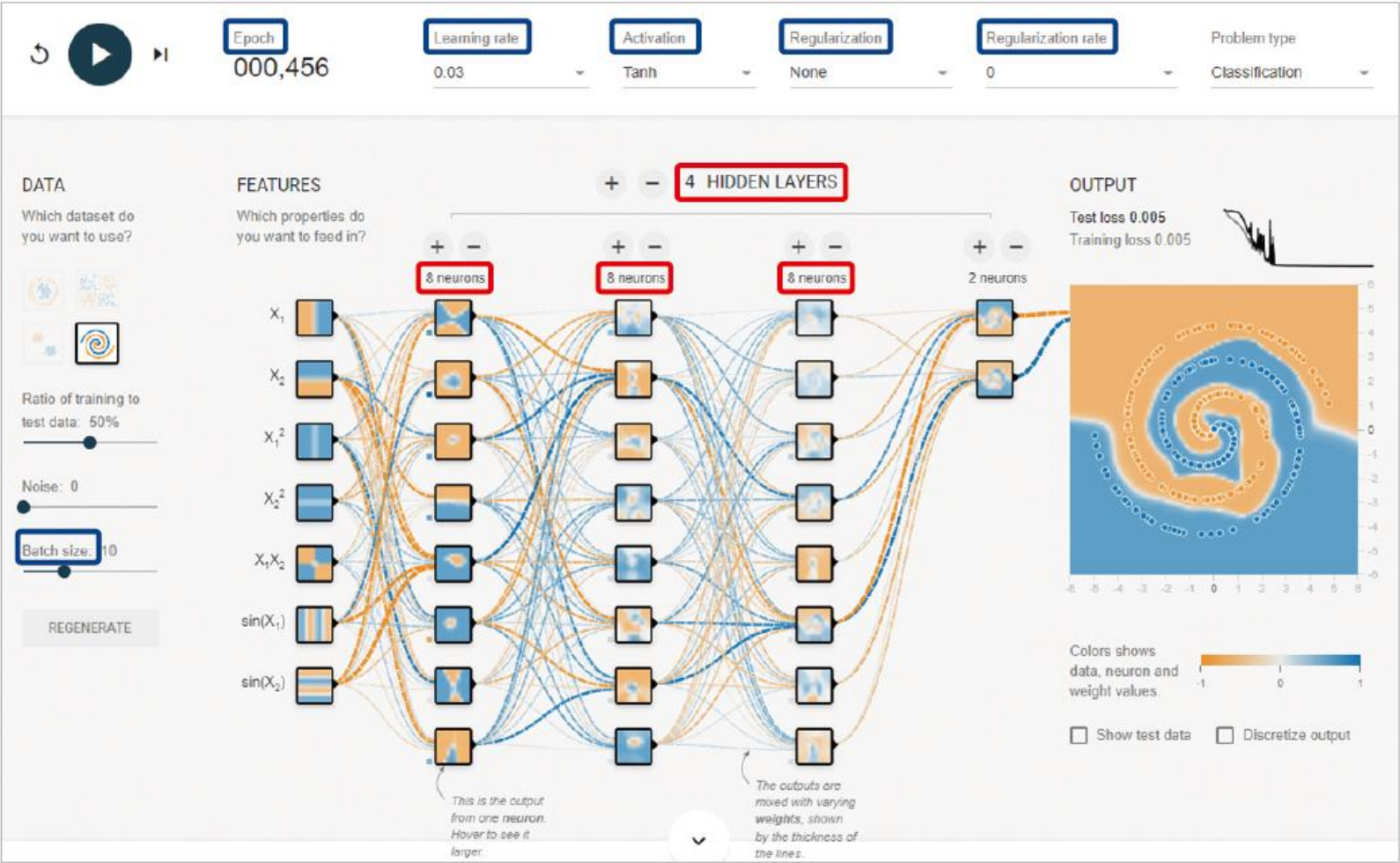
정확률= 98.42000007629395



하이퍼 매개변수 다루기

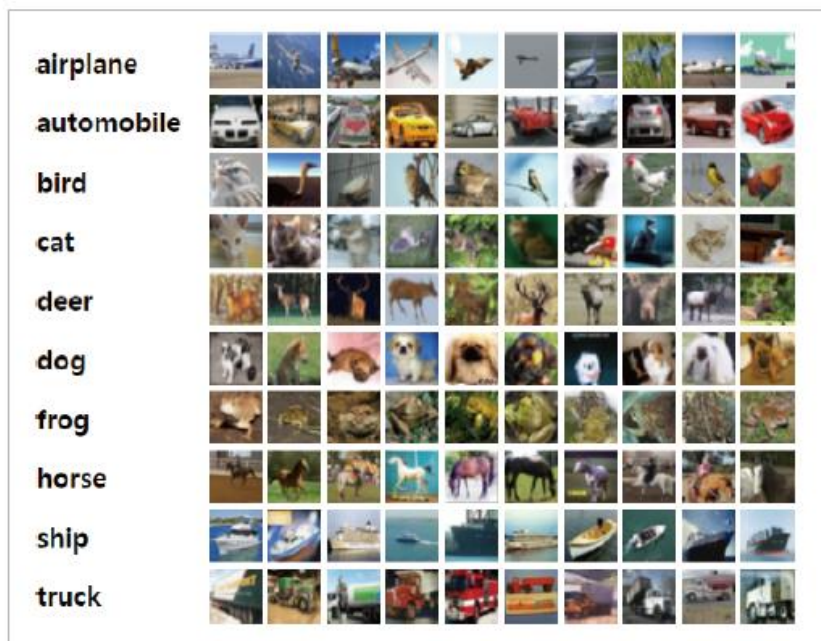
■ 하이퍼 매개변수 설정 체험

파란 박스는 신경망 구조
빨간 박스는 학습에 관련된 하이퍼 매개변수



자연 영상 인식

■ CIFAR-10과 CIFAR-100(장난감 자연 영상 데이터셋)



(a) CIFAR-10

Superclass	Classes
aquatic mammals	beaver, dolphin, otter, seal, whale
fish	aquarium fish, flatfish, ray, shark, trout
flowers	orchids, poppies, roses, sunflowers, tulips
food containers	bottles, bowls, cans, cups, plates
fruit and vegetables	apples, mushrooms, oranges, pears, sweet peppers
household electrical devices	clock, computer keyboard, lamp, telephone, television
household furniture	bed, chair, couch, table, wardrobe
insects	bee, beetle, butterfly, caterpillar, cockroach
large carnivores	bear, leopard, lion, tiger, wolf
large man-made outdoor things	bridge, castle, house, road, skyscraper
large natural outdoor scenes	cloud, forest, mountain, plain, sea
large omnivores and herbivores	camel, cattle, chimpanzee, elephant, kangaroo
medium-sized mammals	fox, porcupine, possum, raccoon, skunk
non-insect invertebrates	crab, lobster, snail, spider, worm
people	baby, boy, girl, man, woman
reptiles	crocodile, dinosaur, lizard, snake, turtle
small mammals	hamster, mouse, rabbit, shrew, squirrel
trees	maple, oak, palm, pine, willow
vehicles 1	bicycle, bus, motorcycle, pickup truck, train
vehicles 2	lawn-mower, rocket, streetcar, tank, tractor

(b) CIFAR-100

인식 실험에 사용할 자연 영상 데이터셋

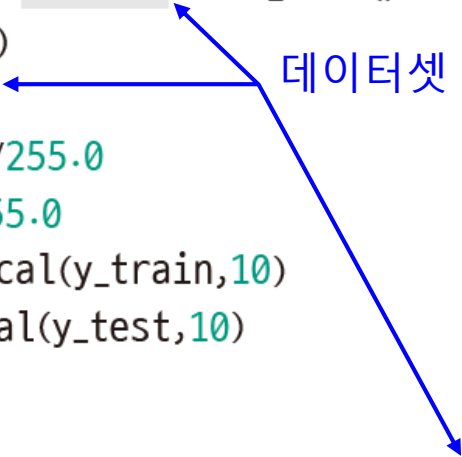
자연 영상 인식

깊은 다층 퍼셉트론으로 CIFAR-10 인식하기

01~07행은 [프로그램 7-5]와 같음

```
08 ... ..
09 (x_train,y_train),(x_test,y_test)=ds.cifar10.load_data()
10 x_train=x_train.reshape(50000,3072)
11 x_test=x_test.reshape(10000,3072)
12 x_train=x_train.astype(np.float32)/255.0
13 x_test=x_test.astype(np.float32)/255.0
14 y_train=tf.keras.utils.to_categorical(y_train,10)
15 y_test=tf.keras.utils.to_categorical(y_test,10)
16
17 dmlp=Sequential()
18 dmlp.add(Dense(units=1024,activation='relu',input_shape=(3072,)))
19 dmlp.add(Dense(units=512,activation='relu'))
20 dmlp.add(Dense(units=512,activation='relu'))
21 dmlp.add(Dense(units=10,activation='softmax'))
...
```

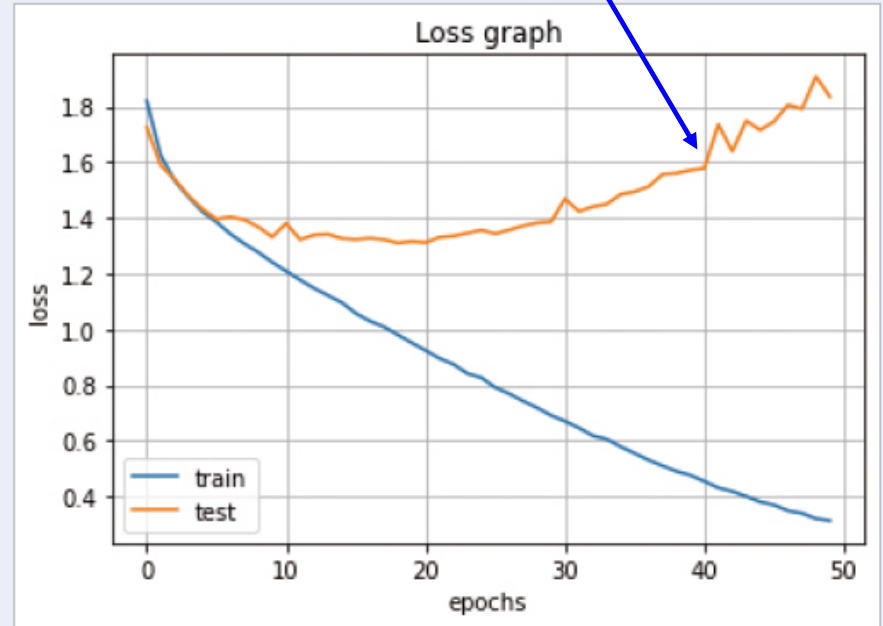
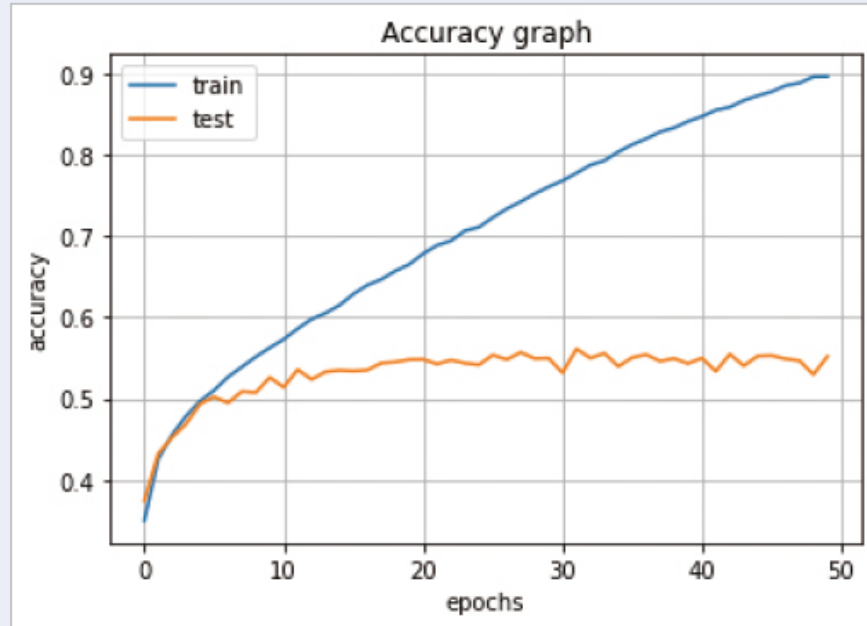
데이터셋 관련 부분만 수정



자연 영상 인식

과잉 적합

정확률= 55.15999794006348



정리

정리

- 컴퓨터 비전
- 딥러닝 비전